



兰州工业学院
LANZHOU INSTITUTE OF TECHNOLOGY



第3章 PyTorch深度学习基础



3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

- 1. LeNet5
- 2. AlexNet
- 3. VGGNet
- 4. GoogLeNet
- 5. ResNet

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

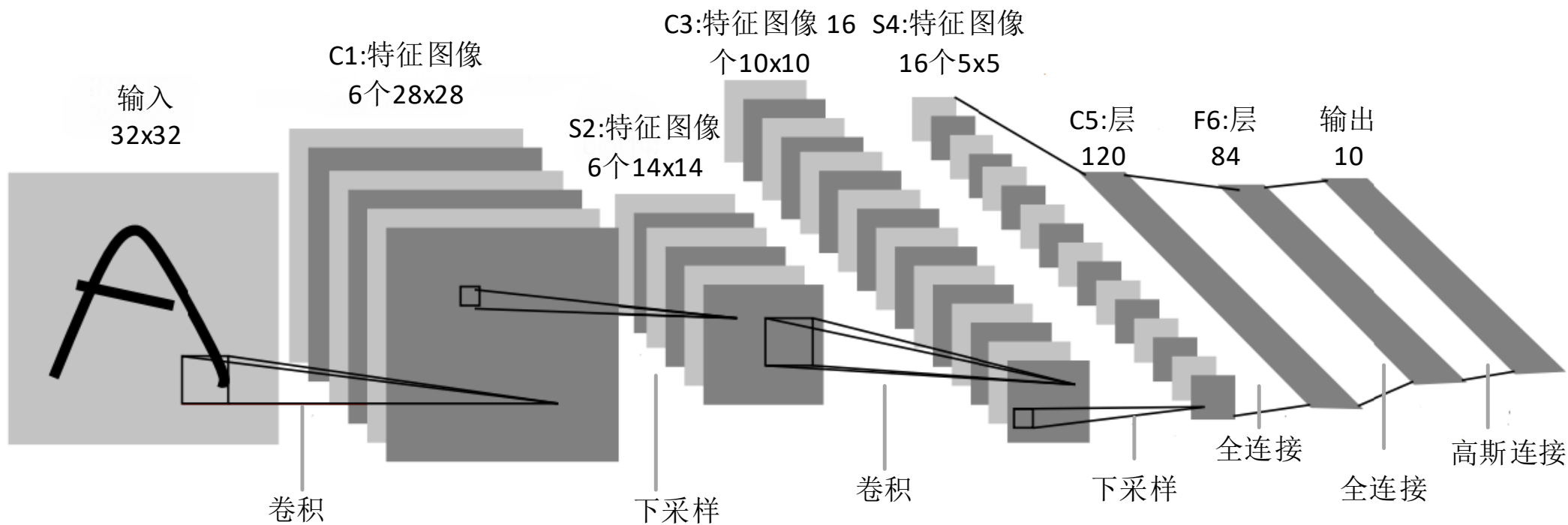
➤ 1. LeNet5

LeNet5是杨立昆（Yann LeCun）在1998年设计的用于手写数字识别的卷积神经网络，当年大多数银行就是用LeNet5来识别支票上面的手写数字的，是早期卷积神经网络中最有代表性的实验系统之一。LeNet5共有7层（不包括输入层），每层都包含不同数量的训练参数，其网络结构如图所示。

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

➤ 1. LeNet5



3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

➤ 1. LeNet5

- ◆ 输入层: 灰度图, $32 \times 32 \times 1$,
- ◆ 第一卷积层: 6个大小为 $5 \times 5 \times 1$ 的卷积核, 输出深度为6,
参数: $(5 \times 5 + 1) \times 6$, 输出特征图大小: $32 - 5 + 1 = 28$, $28 \times 28 \times 6$
- ◆ 第一池化层: 6个 2×2 卷积核, 步长2, 输出为 $14 \times 14 \times 6$, 不消耗参数
- ◆ 第二卷积层: 16个 $5 \times 5 \times 6$ 的卷积核, 无填充, 步长为1, 输出特征图大小:
 $10 \times 10 \times 16$, 参数 $(5 \times 5 \times 6 + 1) \times 16 = 2416$

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

➤ 1. LeNet5

◆ **第二池化层**: 16个 2×2 卷积核, 步长为2, $5 \times 5 \times 16$, 不消耗参数

◆ **两个全连接层**: 拉直成一维向量, 长度: $5 \times 5 \times 16 = 400$, 作为第一个全连接层 (包含120个神经元)的输入, 输出节点个数为120。第二个全连接层有84个神经元, 第一个参数: $(400+1) \times 120 = 48120$, 第二个参数: $(120+1) \times 84 = 10164$

◆ **输出层**: 10个神经元, 将输入张量通过线性映射, 到0~9这10个分类上, 参数: $(84+1) \times 10$

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

➤ 1. 构建网络

```
import torch.nn as nn
import torch.nn.functional as F
class LeNet(nn.Module):
    """
    def __init__(self):
        super(LeNet, self).__init__() # 直接继承nn.Module的init
        ## 构建卷积层
        self.conv = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, stride=1, padding=0),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.Conv2d(6, 16, 5),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
```

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

➤ 1. 构建网络

```
import torch.nn as nn
import torch.nn.functional as F
class LeNet(nn.Module):

    def __init__(self):
        super(LeNet, self).__init__() # 直接继承nn.Module的init
        ## 构建全连接层
        self.fc = nn.Sequential(
            nn.Linear(16 * 5 * 5, 120),
            nn.ReLU(),
            nn.Linear(120, 84),
            nn.ReLU(),
            nn.Linear(84, 10)
        )
```

```
def forward(self, x):
    feature = self.conv(x)
    output =
self.fc(feature.view(x.size()[0], -1))
    return output
model = LeNet()
model.to(device)
from torchsummary import summary
IMSIZE = 32
summary(model, (1, 32, 32))
```

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

➤ 1. 构建网络

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 6, 28, 28]	156
ReLU-2	[-1, 6, 28, 28]	0
MaxPool2d-3	[-1, 6, 14, 14]	0
Conv2d-4	[-1, 16, 10, 10]	2,416
ReLU-5	[-1, 16, 10, 10]	0
MaxPool2d-6	[-1, 16, 5, 5]	0
Linear-7	[-1, 120]	48,120
ReLU-8	[-1, 120]	0
Linear-9	[-1, 84]	10,164
ReLU-10	[-1, 84]	0
Linear-11	[-1, 10]	850

Total params: 61,706

Trainable params: 61,706

Non-trainable params: 0

Input size (MB): 0.00

Forward/backward pass size (MB): 0.11

Params size (MB): 0.24

Estimated Total Size (MB): 0.35

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

➤ 2. AlexNet

AlexNet于2012年由阿莱克斯·克里泽夫斯基（Alex Krizhevsky），伊尔亚·苏茨克维（Ilya Sutskever）和杰弗里·辛顿（Geoffrey Hinton）等人提出，并赢得了2012届图像识别大赛的冠军，使得CNN成为在图像分类上的核心算法模型。AlexNet网络一共分为11层，含5个卷积层以及3个全连接层，除此之外还有3个池化层。在每一个卷积层中包含了激活函数RELU以及局部响应归一化（LRN）处理，然后经过降采样（池化处理）。其网络结构如图所示。

3.1 卷积神经网络基础

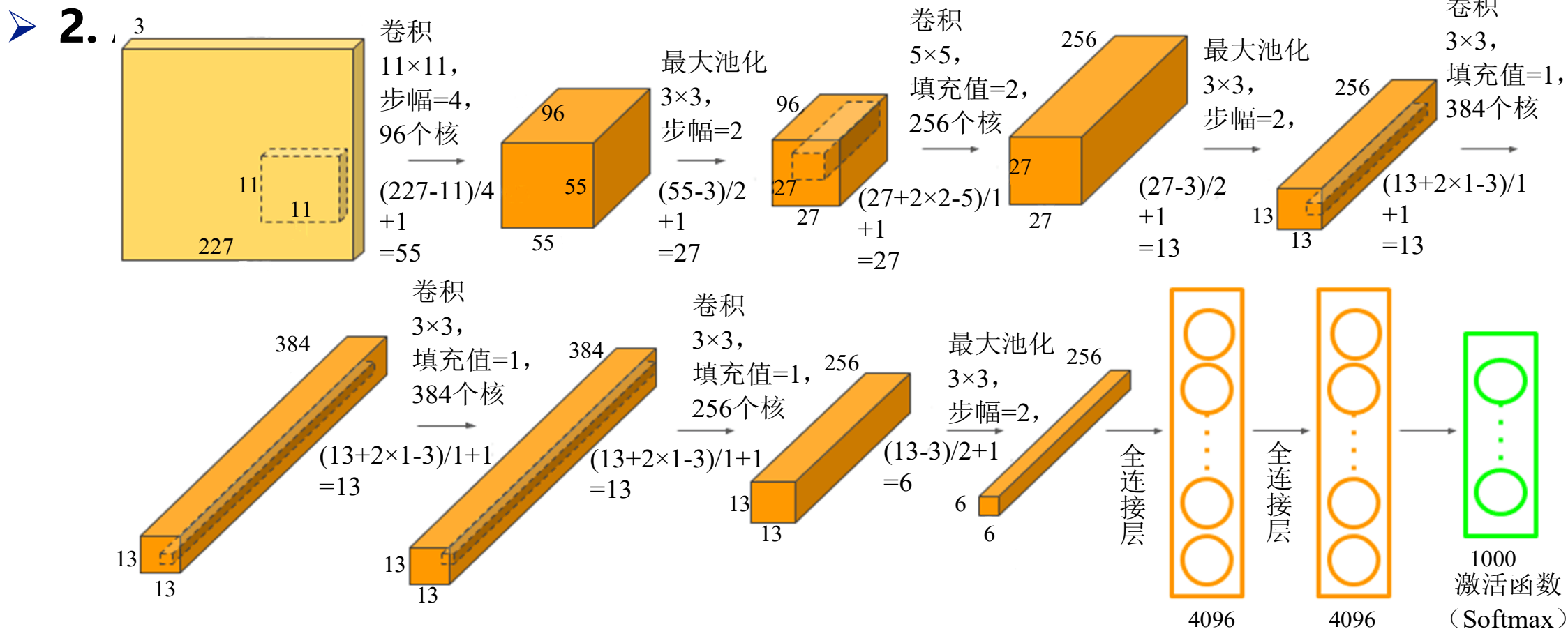
3.1.1 常用卷积神经网络算法及其结构

➤ 2. AlexNet

图中输入的图片尺寸为 256×256 ，然后对输入的图片进行随机裁剪得到 227×227 大小的图片，将图片输入进网络中，最后得到1000个分布区间为 $(0,1)$ 的数的输出，代表输入样本的所属类别的概率。

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构



3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

➤ 2. AlexNet

第 1 层: Conv2D ($11 \times 11, 96$), stride(4), valid, output: $55 \times 55 \times 96$ 。

第 2 层: MaxPooling2D (3×3), stride(2), output: $27 \times 27 \times 96$ 。

第 3 层: Conv2D ($5 \times 5, 256$), same, output: $27 \times 27 \times 256$ 。

第 4 层: MaxPooling2D (3×3), stride(2), output: $13 \times 13 \times 256$ 。

第 5 层: Conv2D ($3 \times 3, 384$), same, output: $13 \times 13 \times 384$ 。

第 6 层: Conv2D ($3 \times 3, 84$), same, output: $13 \times 13 \times 384$ 。

第 7 层: Conv2D ($3 \times 3, 256$), same, output: $13 \times 13 \times 256$ 。

第 8 层: MaxPooling2D (3×3), stride (2), output: $6 \times 6 \times 256$ 。

输出层: Flatten, Dense(4096), Dropout(0.5), Dense (4096), Dropout (0.5), output。output

输出的节点根据真实的应用案例决定。

3.1 卷积神经网络基础

```
class My_AlexNet(nn.Module):
```

```
    def __init__(self):
```

```
        super(My_AlexNet, self).__init__()
```

```
        self.features = nn.Sequential(
```

```
            nn.Conv2d(in_channels=3,out_channels=96,kernel_size=11,stride=4),
```

```
            nn.ReLU(),
```

```
            nn.MaxPool2d(kernel_size=3,stride=2),
```

```
            nn.Conv2d(in_channels=96,out_channels=256,kernel_size=5,padding=2),
```

```
            nn.ReLU(),
```

```
            nn.MaxPool2d(kernel_size=3,stride=2),
```

```
            nn.Conv2d(in_channels=256,out_channels=384,padding=1,kernel_size=3),
```

```
            nn.ReLU(),
```

```
            nn.Conv2d(in_channels=384,out_channels=384,kernel_size=3,padding=1),
```

```
            nn.ReLU(),
```

```
            nn.Conv2d(in_channels=384,out_channels=256,kernel_size=3,padding=1),
```

```
            nn.ReLU(),
```

```
            nn.MaxPool2d(kernel_size=3,stride=2)        )
```

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

```
class My_AlexNet(nn.Module):  
  
    def __init__(self):  
        super(My_AlexNet, self).__init__()  
        # 全连接层  
        self.classifier = nn.Sequential(  
            nn.Linear(in_features=6*6*256,out_features=4096),  
            nn.ReLU(),  
            nn.Dropout(p=0.5),  
            nn.Linear(4096,4096),  
            nn.ReLU(),  
            nn.Linear(4096,5)        )
```

```
def forward(self,x):  
    x = self.features(x)  
    x = torch.flatten(x,1)  
    result = self.classifier(x)  
    return result
```

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

➤ 3. VGGNet

VGGNet (Visual Geometry Group) 于2014年由牛津大学的凯伦·西蒙扬 (Karen Simonyan) 和安德鲁·齐瑟曼 (Andrew Zisserman) 提出, 主要特点是“**简洁与深度**”。简洁是指它的结构一律采用**步长 (stride) 为1的 3×3 的卷积核 (filter)**, 以及**步长为2的 2×2 的最大池化 (MaxPooling)**, 深度则是因为VGGNet能够达到19层。

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

➤ 3. VGGNet

VGGNet一共有6种不同的网络结构，每种结构都含有5组卷积，每组卷积都使用 3×3 的卷积核，每组卷积后进行一个 2×2 的最大池化，然后是3个全连接层。VGGNet的示意图如图所示，其中网络结构D就是著名的VGG16，网络结构E就是著名的VGG19。

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

➤ 3. VGGNet

✓ VGG网络结构：6种

✓ 从左到右深度越来越深

✓ 加粗体表示新增的层

✓ 所有网络结构都包含5组卷积操作，每组卷积包含一定数量的卷积层——可以看作一个五阶段的卷积特征提取。

✓ 获得2014年ILSVRC分类项目第2名

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 权重	11 权重	13 权重	16 权重	16 权重	19 权重
输入(224 x 224 RGB 图片)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64	conv3-64	conv3-64
最大池化层					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128	conv3-128	conv3-128
最大池化层					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
最大池化层					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
最大池化层					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
最大池化层(maxpooling)					
全连接层-4096(FC-4096)					
全连接层-4096(FC-4096)					
全连接层-1000 (FC-1000)					
激活函数 (Softmax)					

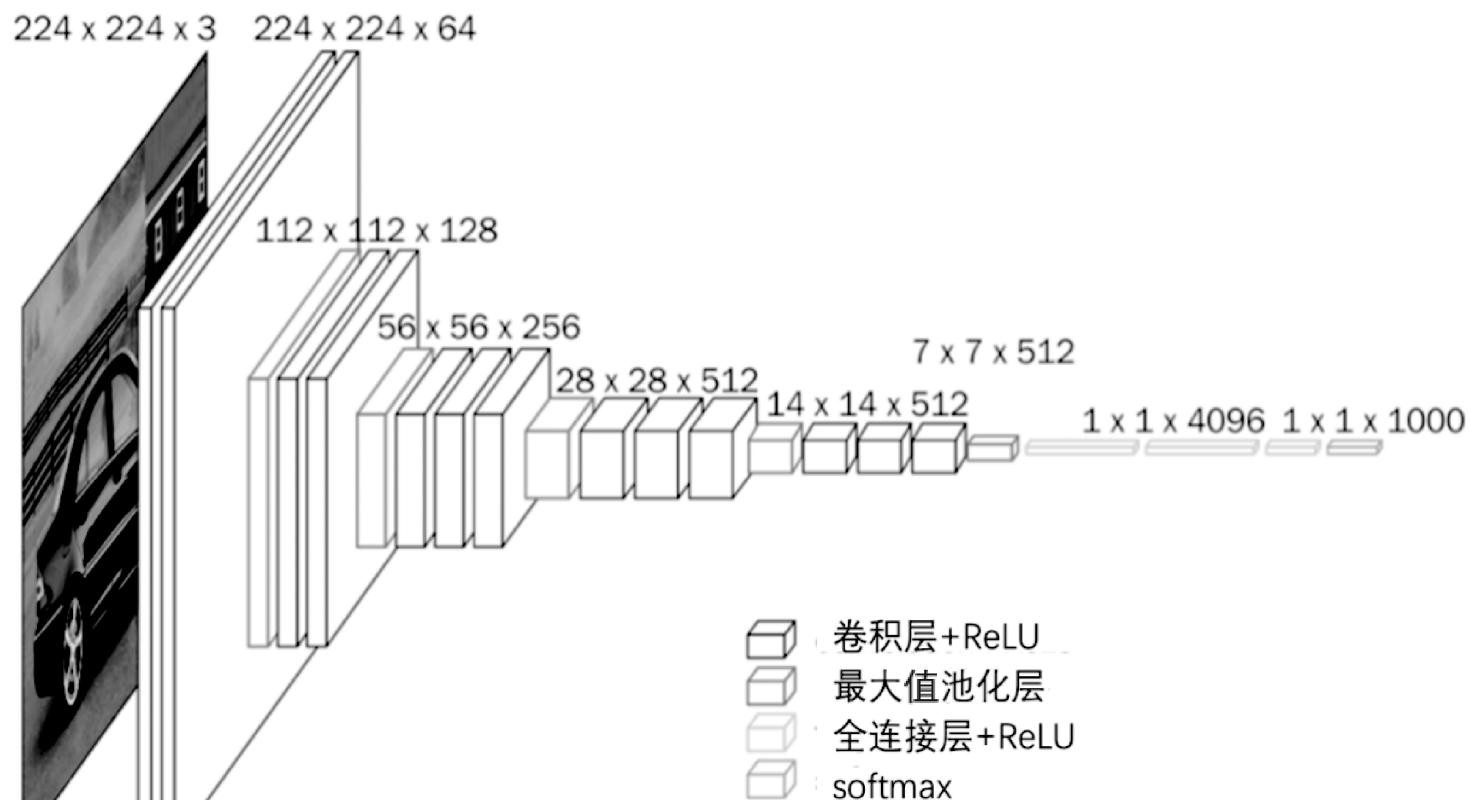
3.1 卷积神经网络基础 (以VGG16为例)

- ✓ 输入层: $224 \times 224 \times 3$ 的彩色图像。
- ✓ 第1组卷积层 (2次卷积) : Conv2D(3×3 , 64), Stride(1), same, ReLU, Output: $224 \times 224 \times 64$ 。
- ✓ 第1个池化层: MaxPooling2D(2×2), Stride(2), Output: $112 \times 112 \times 64$ 。
- ✓ 第2组卷积层 (2次卷积) : Conv2D(3×3 , 128), Stride(1), same, ReLU, Output: $112 \times 112 \times 128$ 。
- ✓ 第2个池化层: MaxPooling2D(2×2), Stride(2), Output: 。
- ✓ 第3组卷积层 (3次卷积) : Conv2D(3×3 , 256), Stride(1), same, ReLU, Output: $56 \times 56 \times 256$ 。
- ✓ 第3个池化层: MaxPooling2D(2×2), Stride(2), Output: 。
- ✓ 第4组卷积层 (3次卷积) : Conv2D(3×3 , 512), Stride(1), same, ReLU, Output: $28 \times 28 \times 512$ 。
- ✓ 第4个池化层: MaxPooling2D(2×2), Stride(2), Output: 。
- ✓ 第5组卷积层 (3次卷积) : Conv2D(3×3 , 512), Stride(1), same, ReLU, Output: 。
- ✓ 第5个池化层: MaxPooling2D(), Stride(2), Output: 。

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

➤ 3. VGG16网络结构：5组卷积组和3个全连接层



3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构



3.

```
class VGG16(nn.Module):
    def __init__(self):
        super(VGG16, self).__init__()
        self.cnn = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64), nn.ReLU(inplace=True),
            nn.Conv2d(64, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64), nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128), nn.ReLU(inplace=True),
            nn.Conv2d(128, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128), nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(128, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256), nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256), nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256), nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(256, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512), nn.ReLU(inplace=True),
            nn.Conv2d(512, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512), nn.ReLU(inplace=True),
            nn.Conv2d(512, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512), nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(512, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512), nn.ReLU(inplace=True),
            nn.Conv2d(512, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512), nn.ReLU(inplace=True),
            nn.Conv2d(512, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512), nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
```

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

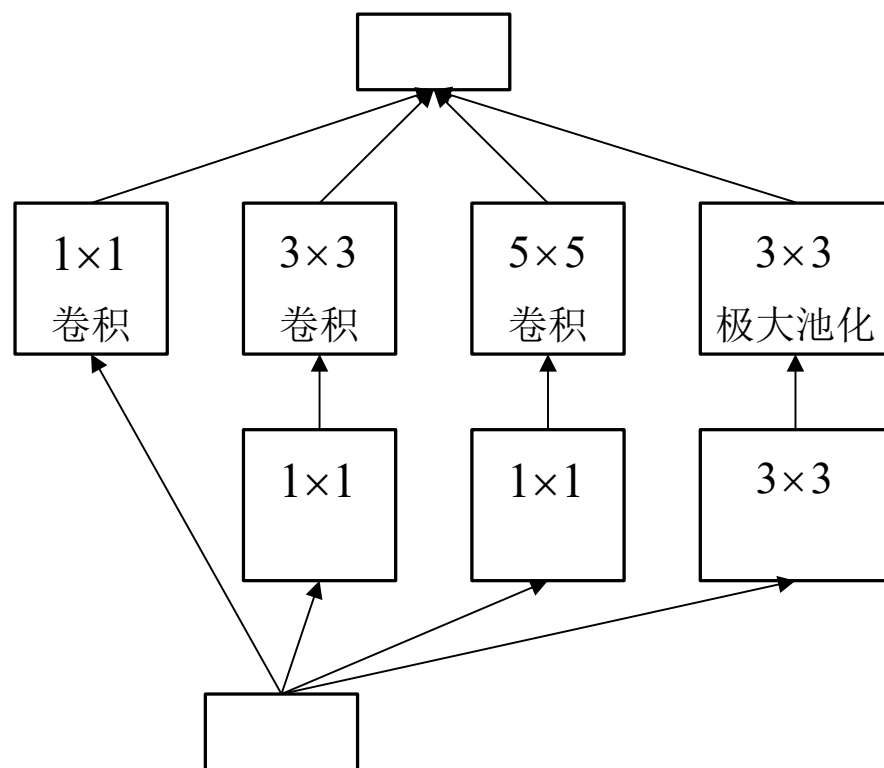
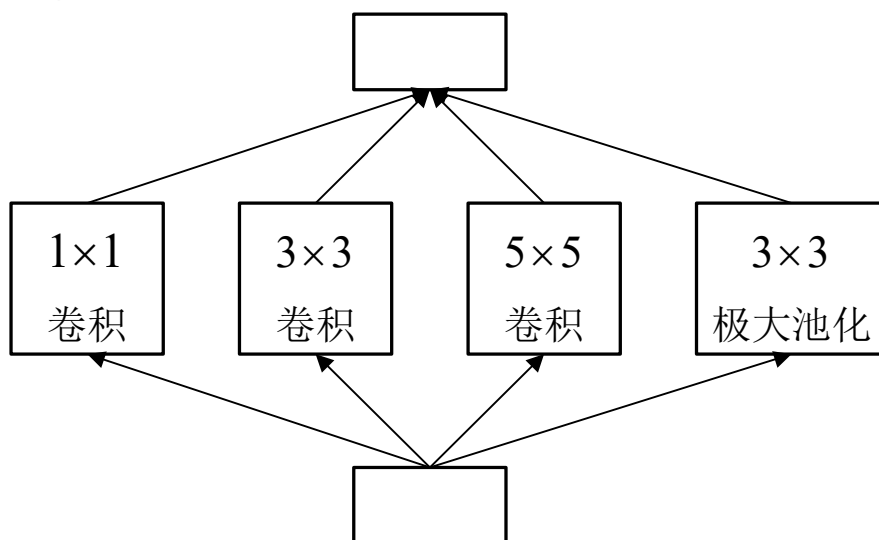
➤ 4. GoogLeNet(Inception)

GoogLeNet是2014年由克里斯提·鲁布托 (Christian Louboutin) 提出的一种全新的深度学习网络结构，在这之前的AlexNet、VGG等结构都是通过增大网络的深度（层数）来获得更好的训练效果，但层数的增加会带来很多副作用，如过拟合、梯度消失、梯度爆炸等。**网络宽度** (inception) 则从另一种角度来提升训练结果，能更高效地利用计算资源，在相同的计算量下能提取到更多的特征。GoogLeNet的inception结构如图所示。

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

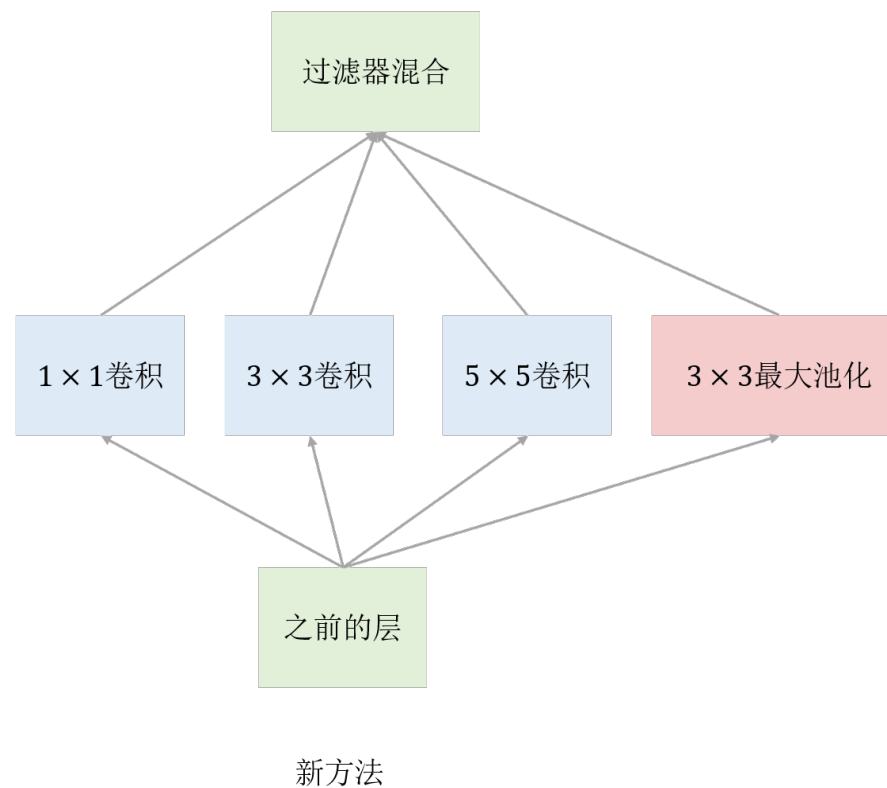
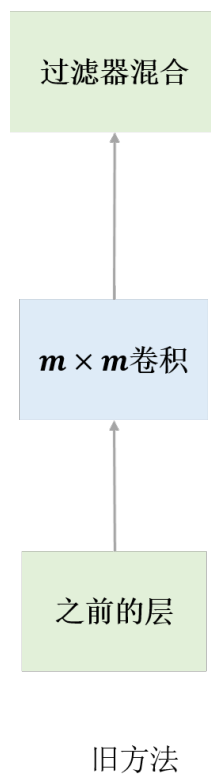
➤ 4. GoogLeNet(Inception)



3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

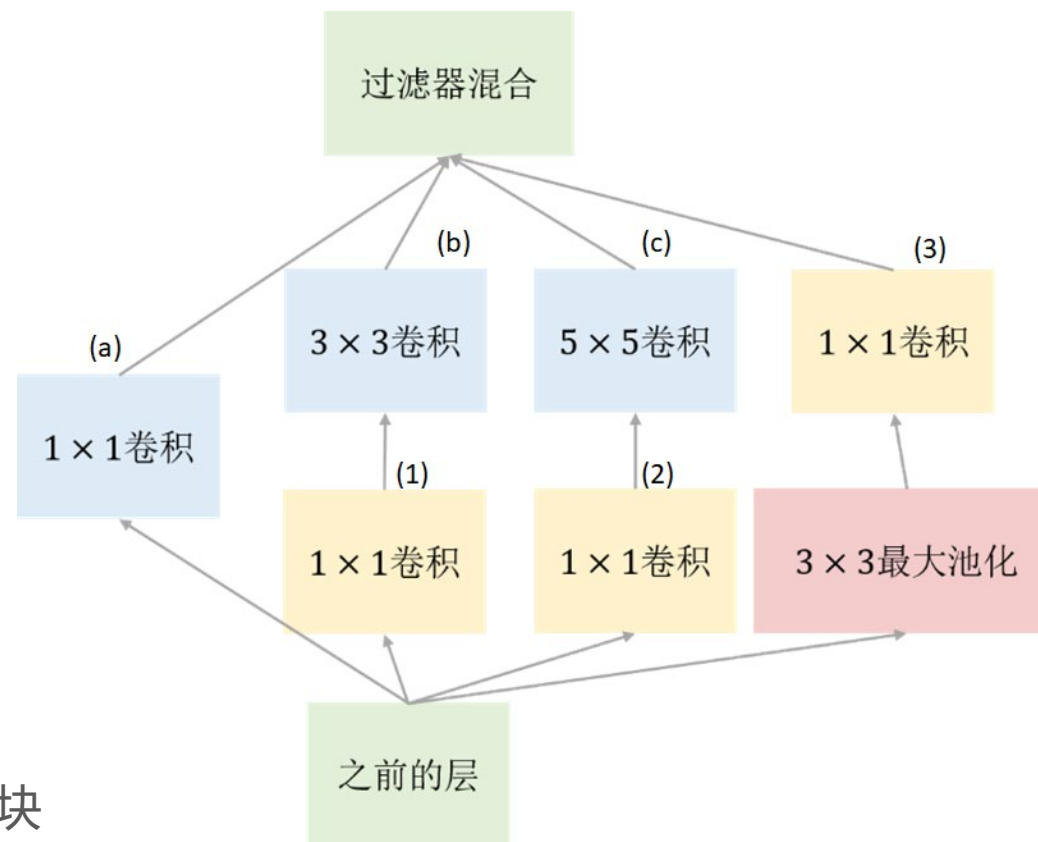
- ✓ 1. Inception V1使用多个不同尺寸的卷积核（直观理解：叠加不同尺寸的卷积核，可以从细节上提取更丰富的特征）。



3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

➤ 4. GoogLeNet(Inception)



Inception V1基础模块

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

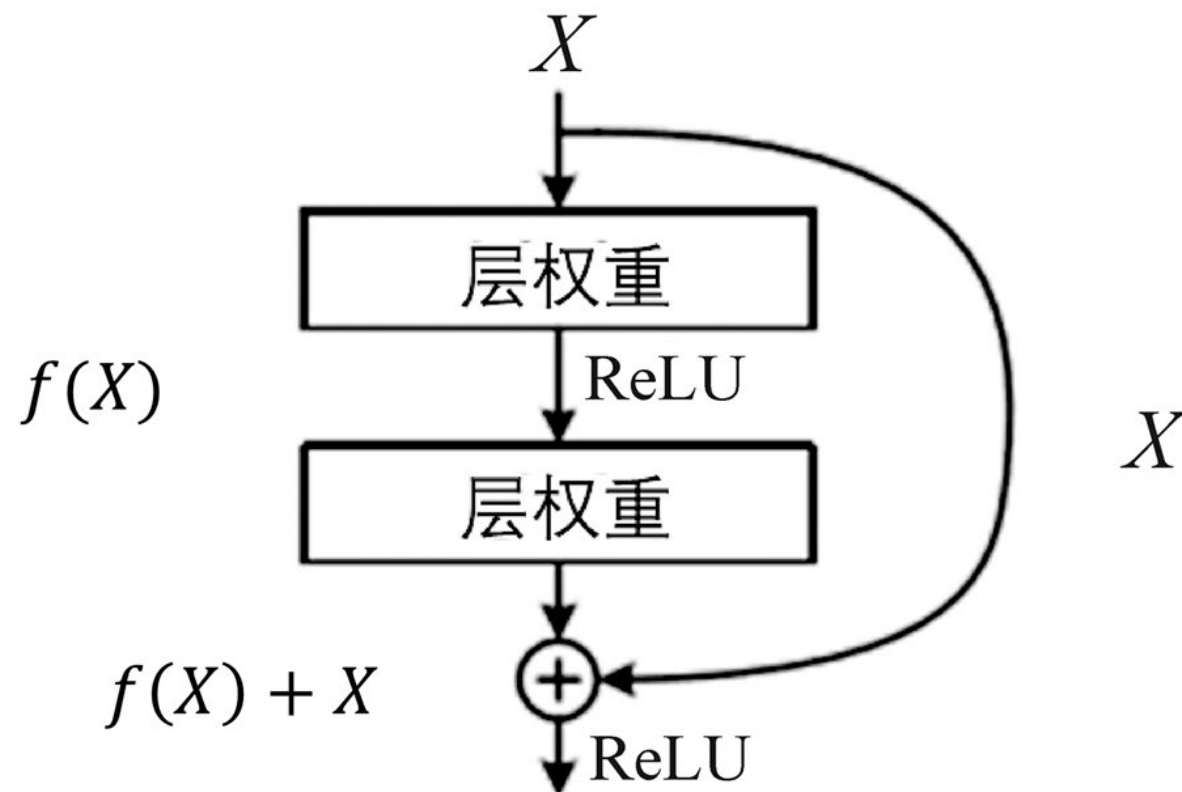
➤ 5. ResNet

随着层数的增加，CNN的效果会遇到瓶颈，甚至会不增反降。ResNet于2015年由微软亚洲研究院的学者们提出，主要是为了解决这个问题，帮助训练更深的网络。ResNet引入了一个残差块（residual block）的结构，如图所示。

3.1 卷积神经网络基础

3.1.1 常用卷积神经网络算法及其结构

➤ 5. ResNet



3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

- 1. 卷积层
- 2. 池化层
- 3. 全连接层
- 4. 归一化层
- 5. 丢弃层

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 1. 卷积层

卷积神经网络的卷积层由若干卷积单元组成，反向传播算法会对每个卷积单元的参数做优化处理。卷积运算的目的是提取输入的不同特征，第一层卷积层只能提取一些简单的特征如边缘、线条和角等，后续更深层的网络能从简单特征中迭代提取更为复杂的特征。下面将介绍卷积层的局部连接和权值共享两个基本特性，然后介绍卷积的实现过程。

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

- ✓ **卷积 (Convolutional)** 是一种特殊的线性运算，用来代替一般的矩阵乘法运算。
- ✓ 卷积运算由以下两个关键参数定义：
 - ✓ 卷积核的大小
 - ✓ 输出特征图的深度：由使用的卷积核的数量决定

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

- ✓ 卷积核从原始图像上左上角第一处 3×3 的区域开始扫描做内积运算，即对应元素相乘并求和，输出的结果为特征图左上角第一个像素值

$1_{\times 1}$	$1_{\times 0}$	$1_{\times 1}$	0	0
$0_{\times 0}$	$1_{\times 1}$	$1_{\times 0}$	1	0
$0_{\times 1}$	$0_{\times 0}$	$1_{\times 1}$	1	1
0	0	1	1	0
0	1	1	0	0

原始图像

1	0	1
0	1	0
1	0	1

卷积核

4		

特征图

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

- ✓ 卷积核向右滑动一格，扫描原始图像第二处 3×3 的区域，再次做内积运算，输出的结果为特征图(0,1)位置的像素值

1	$1_{\times 1}$	$1_{\times 0}$	$0_{\times 1}$	0
0	$1_{\times 0}$	$1_{\times 1}$	$1_{\times 0}$	0
0	$0_{\times 1}$	$1_{\times 0}$	$1_{\times 1}$	1
0	0	1	1	0
0	1	1	0	0

原始图像

1	0	1
0	1	0
1	0	1

卷积核

4	3	

特征图

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

- ✓ 卷积核已经完成对原始图像第一行的扫描，现在开始向下滑动一格，对第二行第一处 3×3 的区域做内积运算，输出的结果为特征图(1,0)位置的像素值

1	1	1	0	0
$0_{\times 1}$	$1_{\times 0}$	$1_{\times 1}$	1	0
$0_{\times 0}$	$0_{\times 1}$	$1_{\times 0}$	1	1
$0_{\times 1}$	$0_{\times 0}$	$1_{\times 1}$	1	0
0	1	1	0	0

原始图像

1	0	1
0	1	0
1	0	1

卷积核

4	3	4
2		

特征图

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

- ✓ 卷积核不断地由左向右由上向下滑动，在滑动过程中与原始图像的对应区域做线性运算从而得到特征图对应位置的像素值，卷积核对原始图像的扫描直到将原始图像全部覆盖为止。在本例中得到的最终结果如图

1	1	1	0	0
0	1	1	1	0
0	0	$1_{\times 1}$	$1_{\times 0}$	$1_{\times 1}$
0	0	$1_{\times 0}$	$1_{\times 1}$	$0_{\times 0}$
0	1	$1_{\times 1}$	$0_{\times 0}$	$0_{\times 1}$

原始图像

1	0	1
0	1	0
1	0	1

卷积核

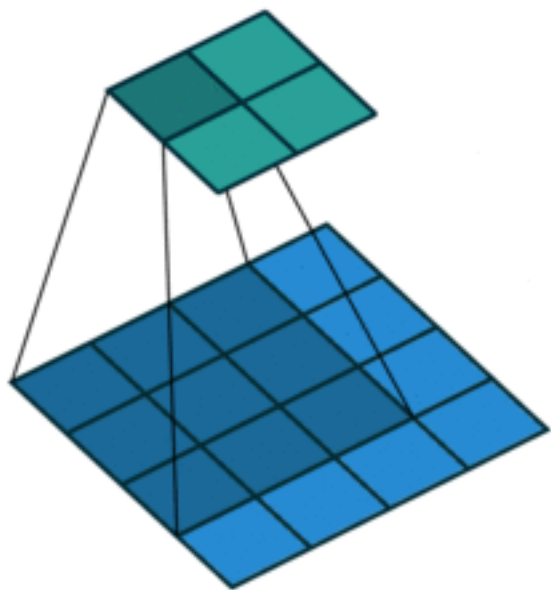
4	3	4
2	4	3
2	3	4

特征图

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

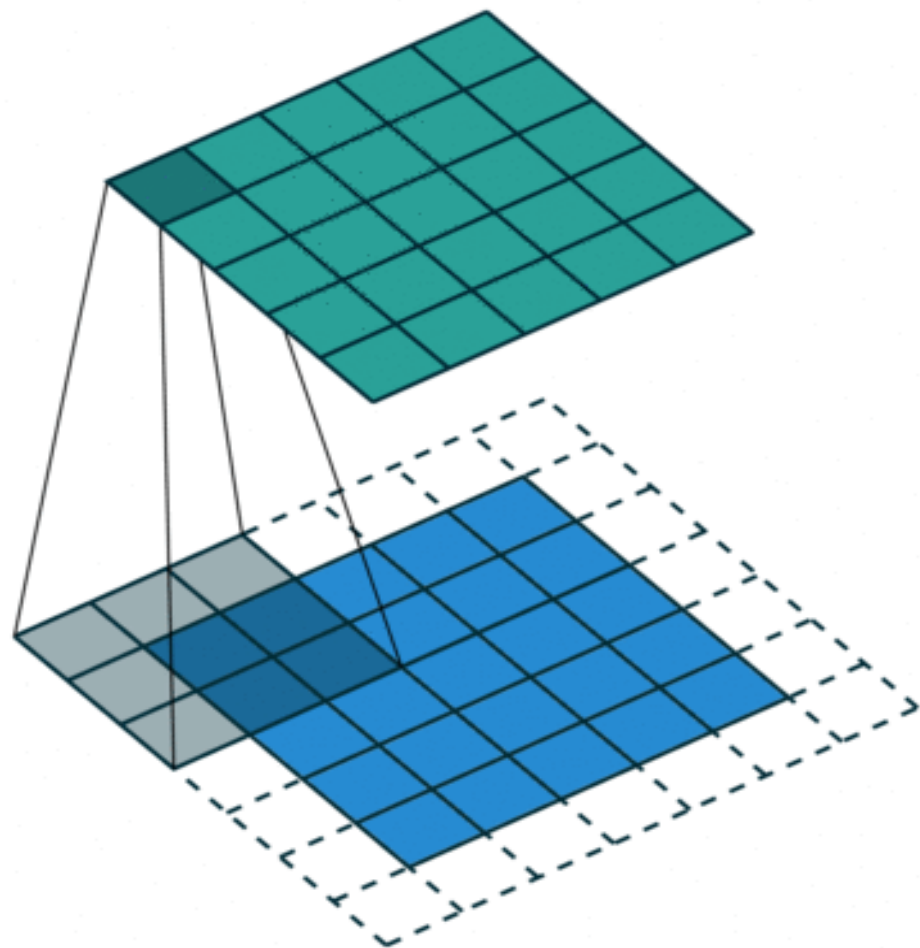
✓ 卷积的三种模式: valid、same、full



- ✓ Valid
- ✓ 滑动步长为 S , 图片大小为 $N1 \times N1$, 卷积核大小为 $N2 \times N2$, 卷积后图像大小: $(N1 - N2) / S + 1 \times (N1 - N2) / S + 1$
- ✓ 该卷积的padding = 0
- ✓ valid卷积, 即仅使用每个卷积的有效部分

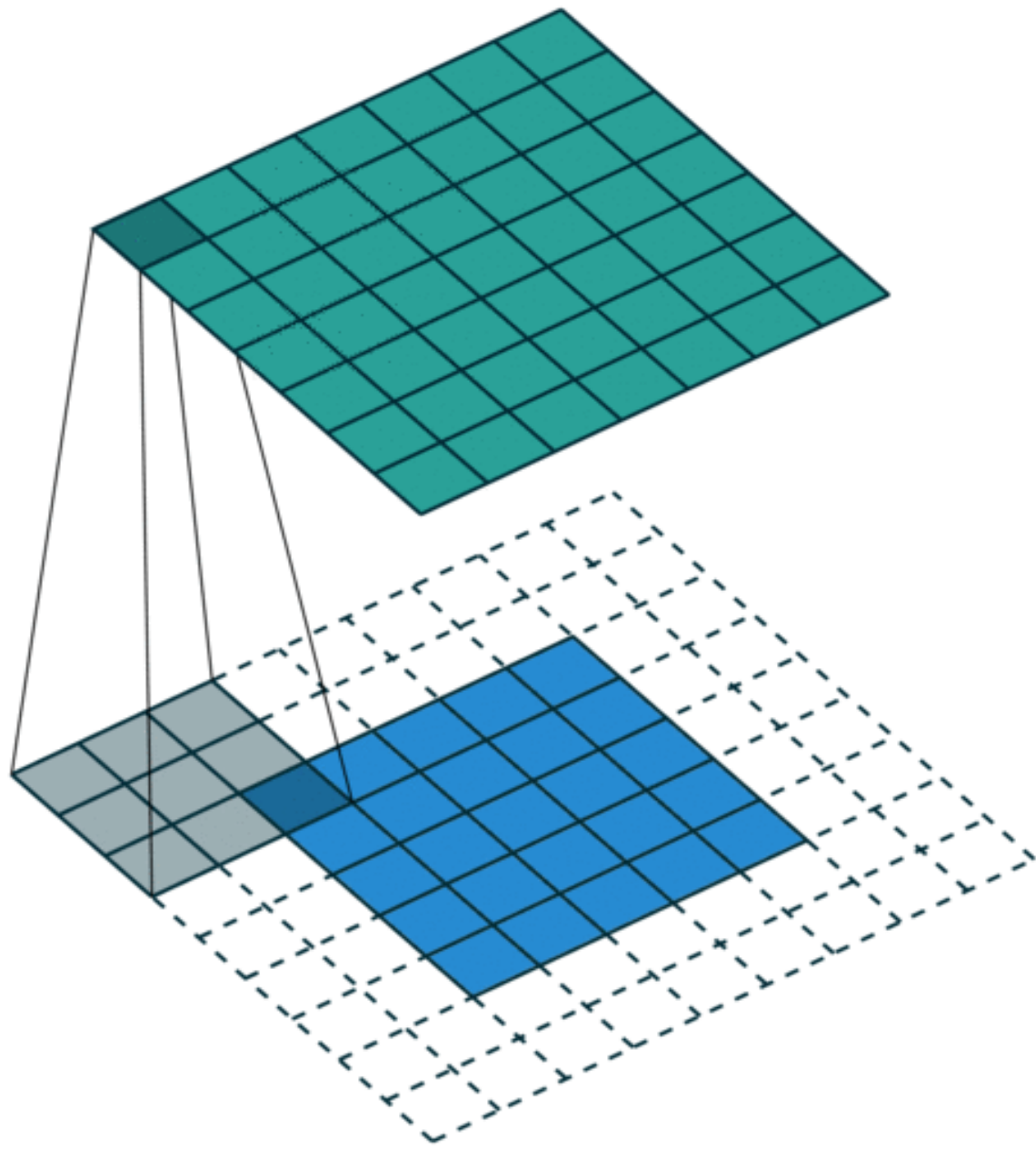
3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层



✓ same

✓ 滑动步长为1，图片大小为 $N_1 \times N_1$ ，卷积核大小为 $N_2 \times N_2$ ，卷积后图像大小： $N_1 \times N_1$



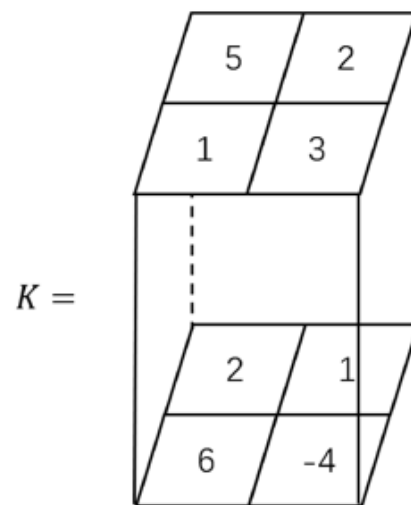
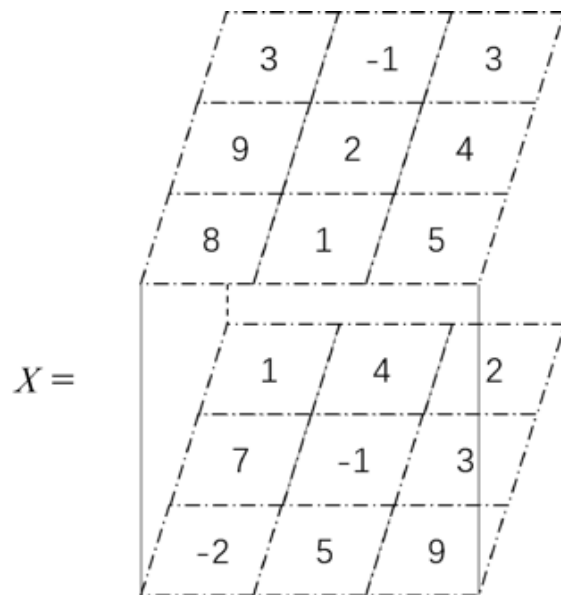
✓ full

✓ 滑动步长为1，图片大小为 $N1 \times N1$ ，卷积核大小为 $N2 \times N2$ ，卷积后图像大小： $(N1 + N2 - 1) \times (N1 + N2 - 1)$

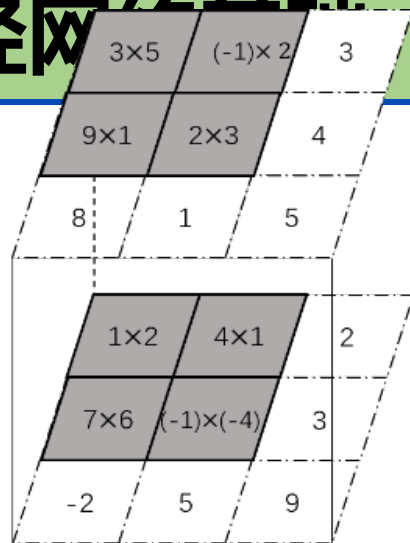
3.1 卷积神经网络基础

3.1.2 卷积层(多通道卷积)

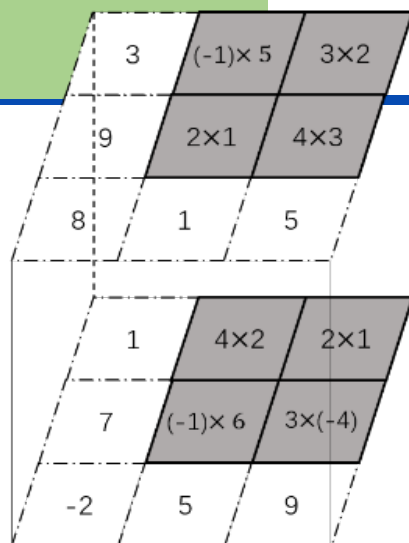
✓ 1个3行3列2深度的三维张量 X 与1个2行2列2深度的三维卷积核 K 的valid卷积



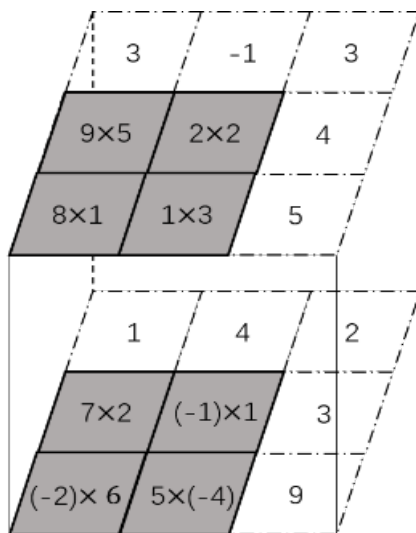
3.1 卷积神经网络基础



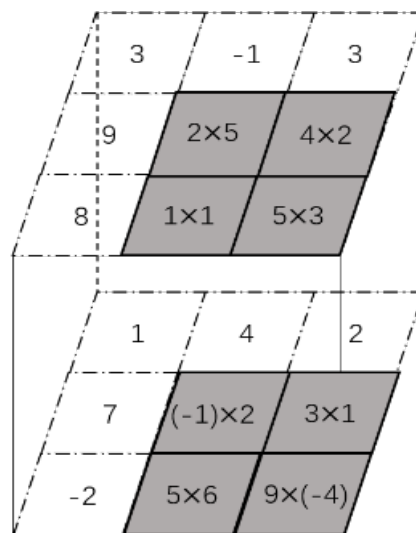
$$3 \times 5 + (-1) \times 2 + 9 \times 1 + 2 \times 3 + 1 \times 2 + 4 \times 1 + 7 \times 6 + (-1) \times (-4) = 80$$



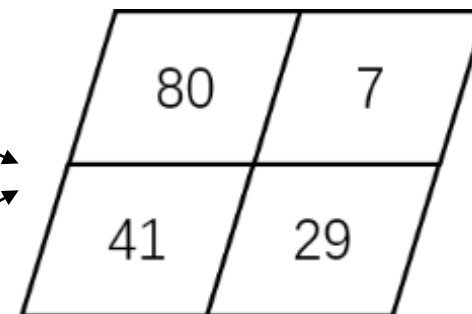
$$(-1) \times 5 + 3 \times 2 + 2 \times 1 + 4 \times 3 + 4 \times 2 + 2 \times 1 + (-1) \times 6 + 3 \times (-4) = 7$$



$$9 \times 5 + 2 \times 2 + 8 \times 1 + 1 \times 3 + 7 \times 2 + (-1) \times 1 + (-2) \times 6 + 5 \times (-4) = 41$$

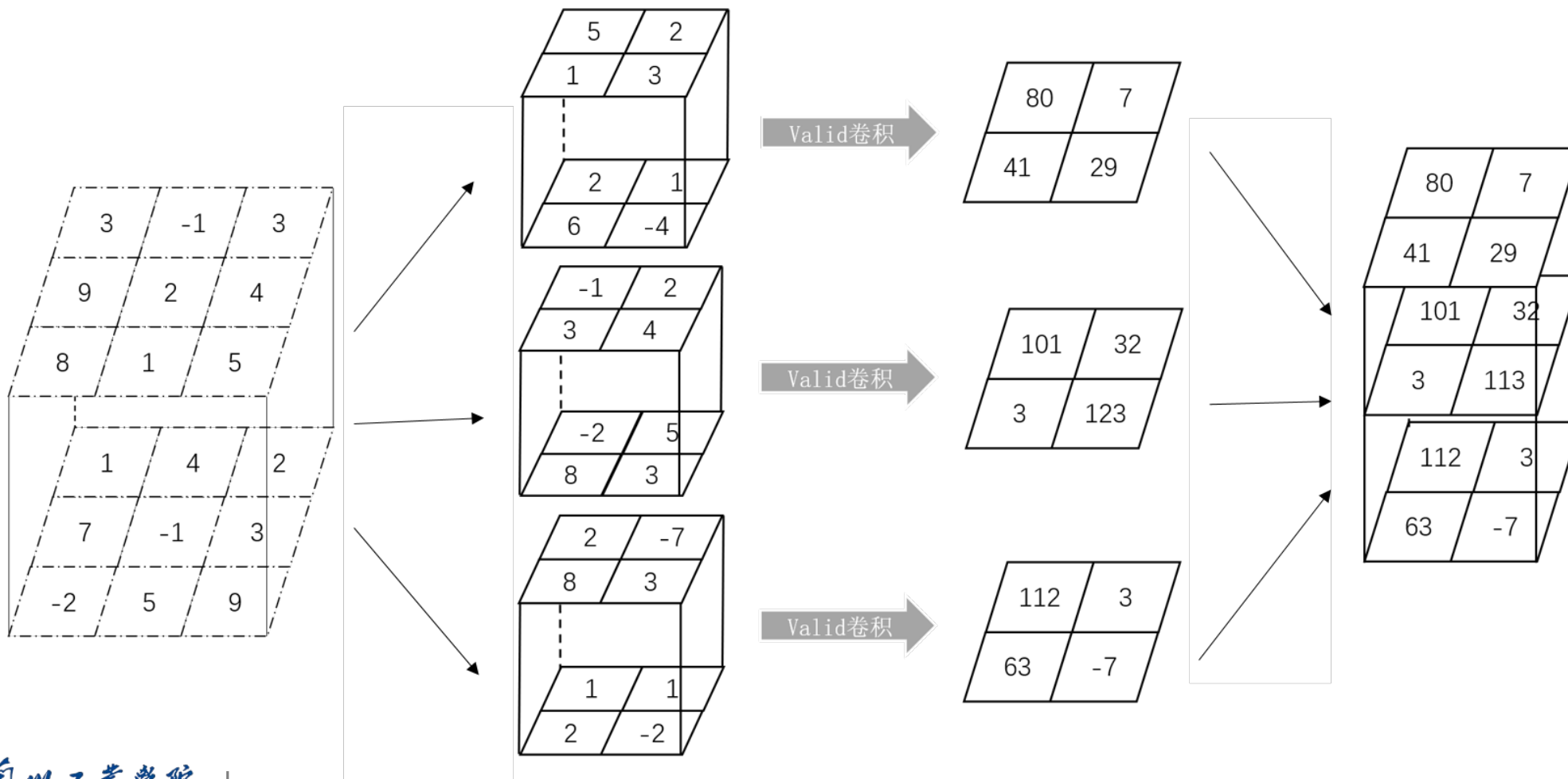


$$2 \times 5 + 4 \times 2 + 1 \times 1 + 5 \times 3 + (-1) \times 2 + 3 \times 1 + 5 \times 6 + 9 \times (-4) = 29$$



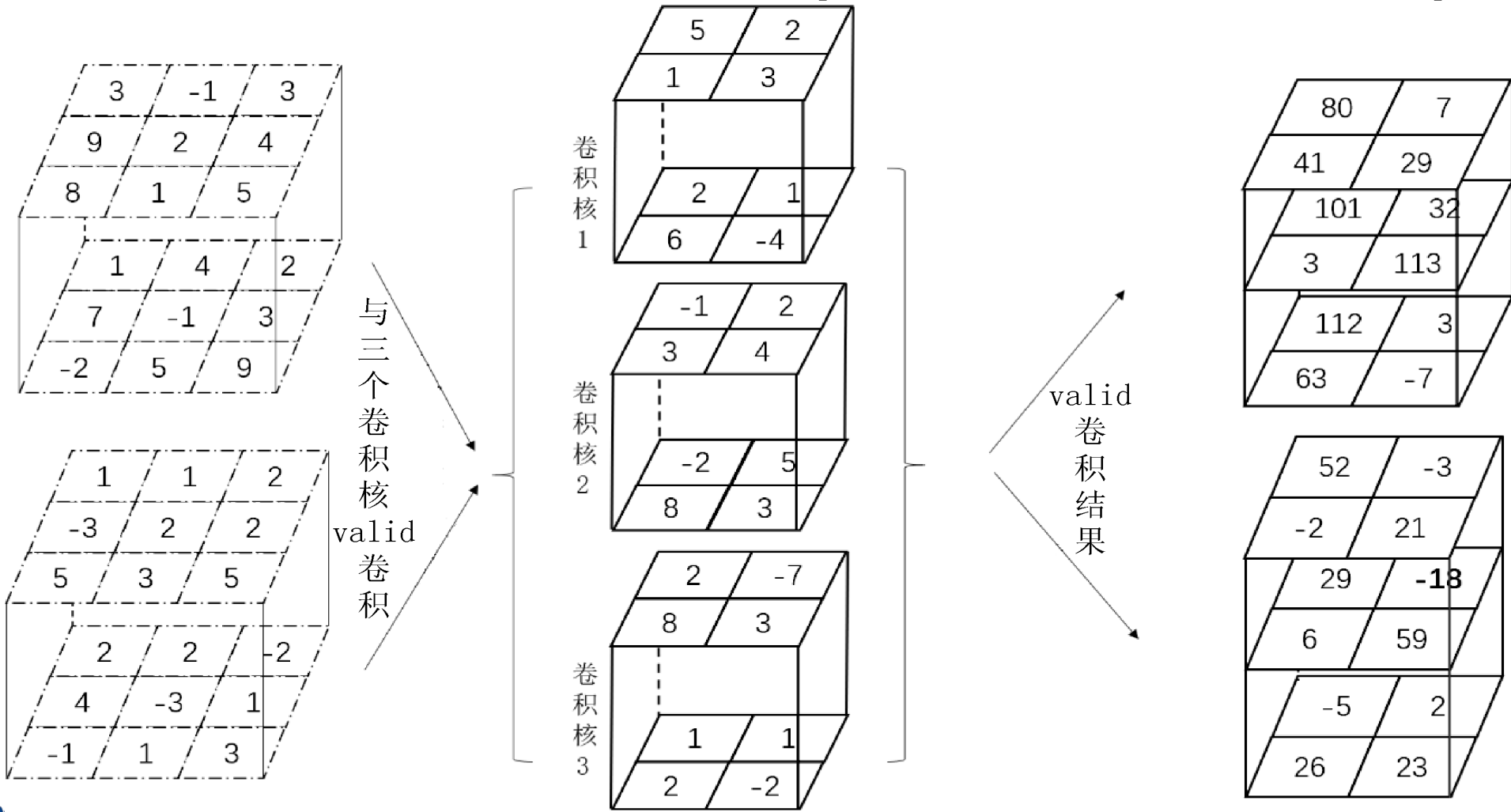
3.1 卷积神经网络基础

3.1.2 单个张量与多个卷积核的卷积(1个 $3 \times 3 \times 2$ 和3个 $2 \times 2 \times 2$)



3.1 卷积神经网络基础

3.1.2 多个张量与多个卷积核的卷积(2个 $3 \times 3 \times 2$ 和3个 $2 \times 2 \times 2$)



3.1 卷积神经网络基础

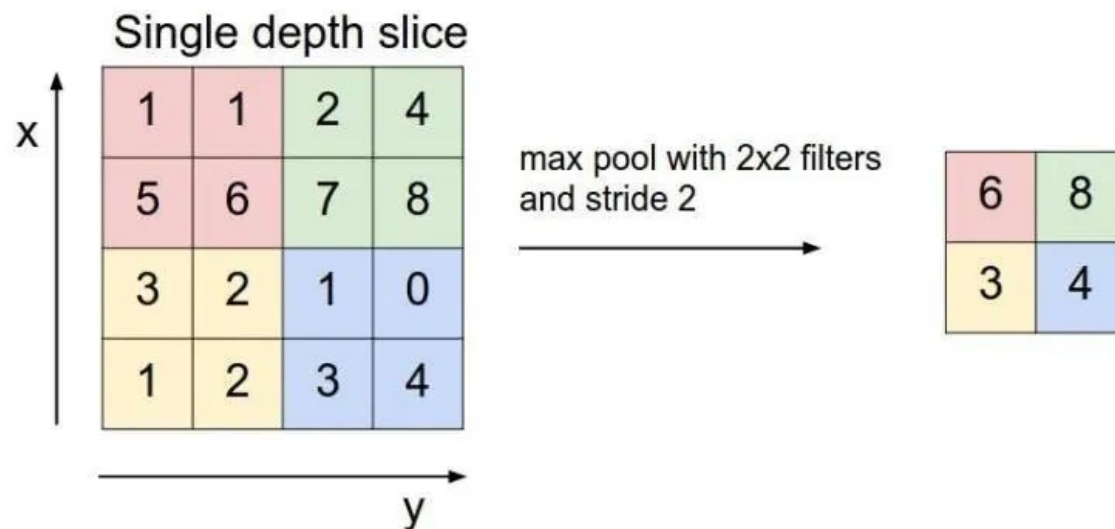
3.1.2 卷积神经网络中的常用网络层

✓ **池化 (Pooling) 操作**是对卷积得到的结果进一步处理，它是将平面内某一位置及其相邻位置的特征值进行统计汇总，并将汇总后的结果作为这一位置在该平面内的值输出，也叫下采样操作。

✓ 分类

✓ 最大值池化

✓ 平均值池化



3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

3	-1	3
9	2	4
8	1	5

9	

(1)

3	-1	3
9	2	4
8	1	5

9	4

(2)

3	-1	3
9	2	4
8	1	5

9	4
9	

(3)

3	-1	3
9	2	4
8	1	5

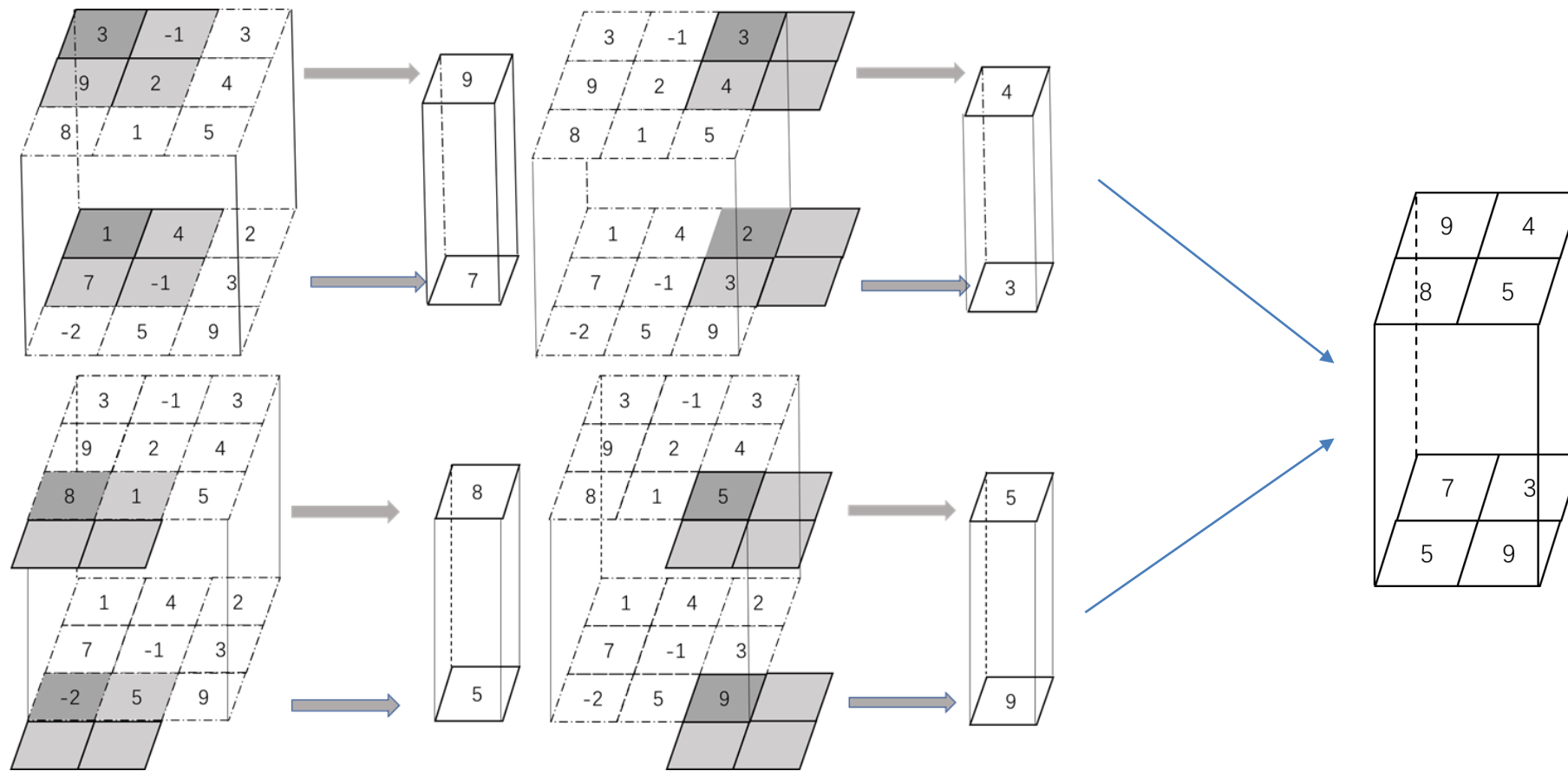
9	4
9	5

(4)

✓ 与卷积区别：池化所用窗口不包含任何参数，

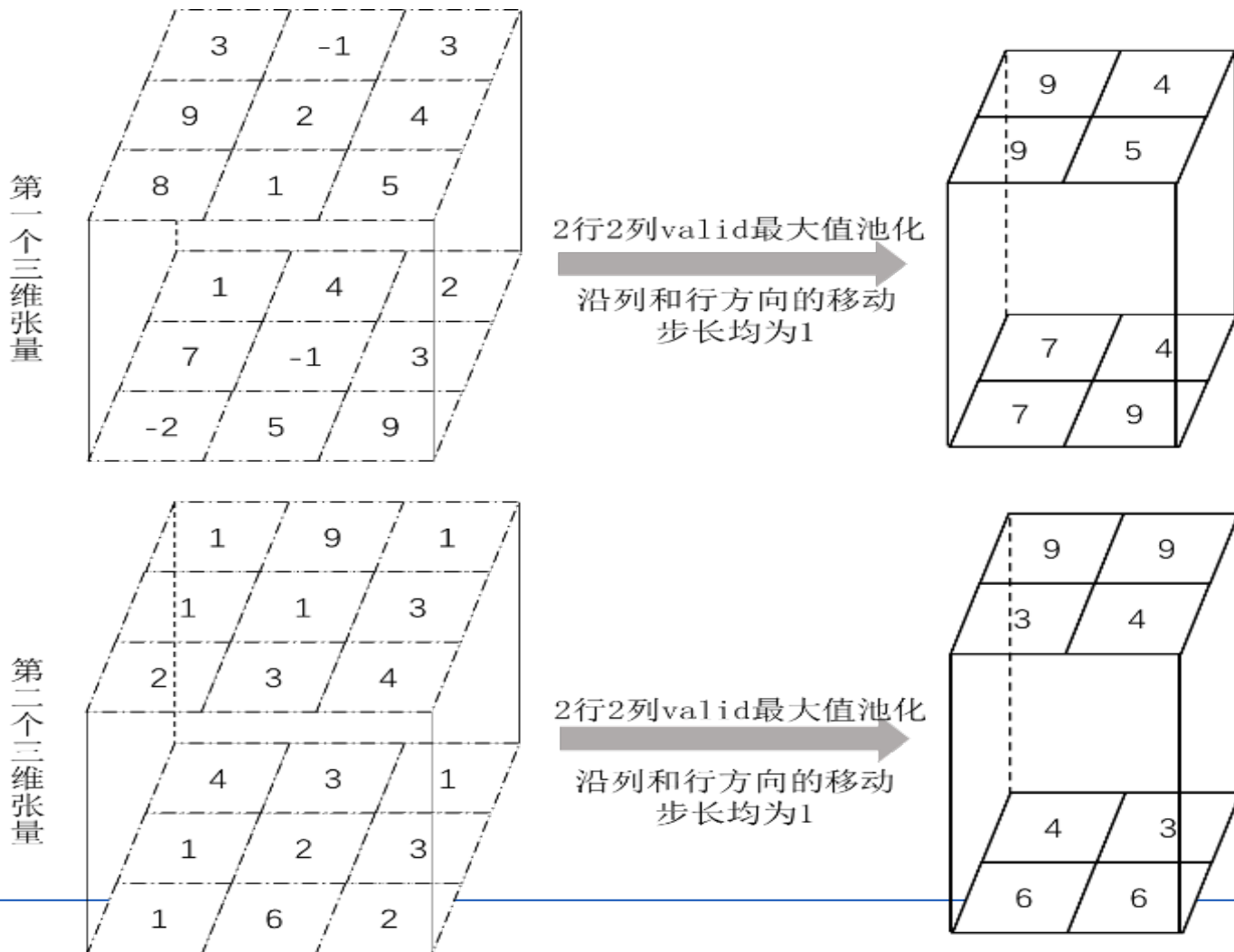
3.1 卷积神经网络基础

3.1.2 多通道池化(一个多通道)



3.1 卷积神经网络基础

3.1.2 多通道池化(多个多通道)



3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 1. 卷积层

(1) PyTorch框架中的常用卷积类型

① Conv2d

Conv2d类的语法格式如下。

`torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True, padding_mode='zeros')`

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 1. 卷积层

(1) PyTorch框架中的常用卷积类型

Conv2d类创建一个卷积核，该卷积核对层输入进行卷积，以生成输出张量。如果参数bias的值为True，则会创建一个偏置向量并将其添加到输出中。

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 1. 卷积层

(1) PyTorch框架中的常用卷积类型

① Conv2d

参数名称	说明
in_channels	接收int，表示输入通道数，无默认值
out_channels	接收int，表示输出通道数，无默认值
kernel_size	接收int或tuple，表示卷积核尺寸，无默认值
stride	接收int或tuple，表示卷积操作的步幅，默认为1
padding	接收int或tuple，表示用于在输入数据中填充的层数，默认为0
dilation	接收int或tuple，表示卷积核内部各点的间距，默认为1
groups	接收int，表示控制输入和输出之间的连接；group=1，输出是所有输入的卷积；group=2，此时相当于有并排的两个卷积层，每个卷积层计算输入通道的一半，并且产生的输出是输出通道的一半，随后将这两个输出连接起来，默认为1
bias	接收bool，为true表示对输出添加可学习的偏置量，默认为True
padding_mode	接收str，表示填充模式，默认为'zeros'

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 1. 卷积层

(1) PyTorch框架中的常用卷积类型

② ConvTranspose2d

转置卷积 (Conv2dTranspose) 在CNN中常用于对特征图进行上采样。

Conv2dTranspose对普通卷积操作中的卷积核做转置处理，将普通卷积的输出作为转置卷积的输入，而转置卷积的输出即普通卷积的输入。

3.1 卷积神经网络基础

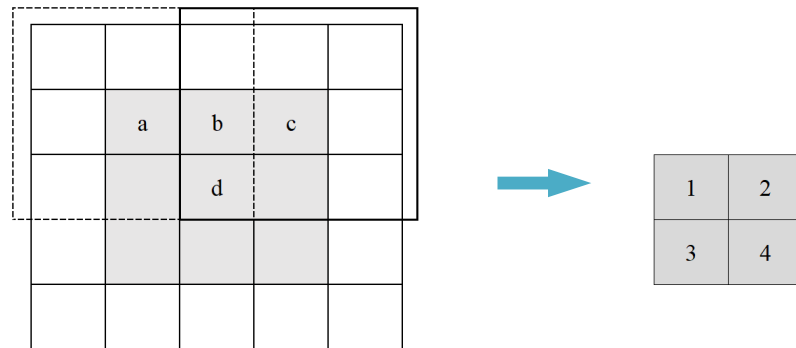
3.1.2 卷积神经网络中的常用网络层

➤ 1. 卷积层

(1) PyTorch框架中的常用卷积类型

② ConvTranspose2d

转置卷积形式上和卷积层的反向梯度计算相同。图中是一个卷积核大小为 3×3 ，步长为2，填充值（padding）为1的普通卷积。卷积核在虚线框位置时输出元素1，在实线框位置时输出元素2。输入元素a仅和输出元素1有运算关系，而输入元素b和输出元素1、2均有关系。同理c只和一个元素2有关，而d和1、2、3和4四个元素都有关。在进行转置卷积时，依然应该保持这个连接关系不变。



3.1 卷积神经网络基础

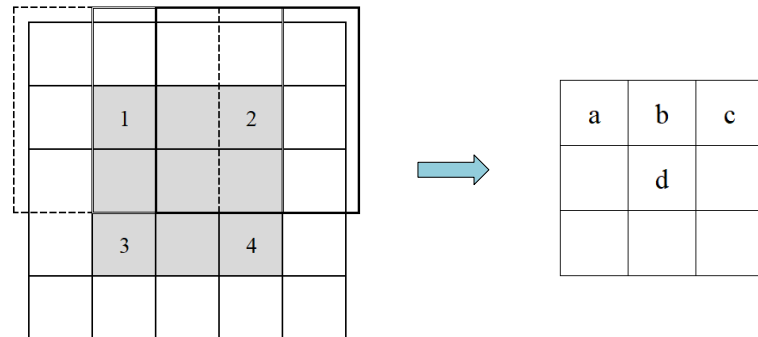
3.1.2 卷积神经网络中的常用网络层

➤ 1. 卷积层

(4) PyTorch框架中的常用卷积类型

② ConvTranspose2d

需要将上页图中右边的特征图作为输入，左边中间的特征图作为输出，并且保证连接关系不变。即a只和1有关，b和1与2两个元素有关，其他以此类推。



3.1 卷积神经网络基础

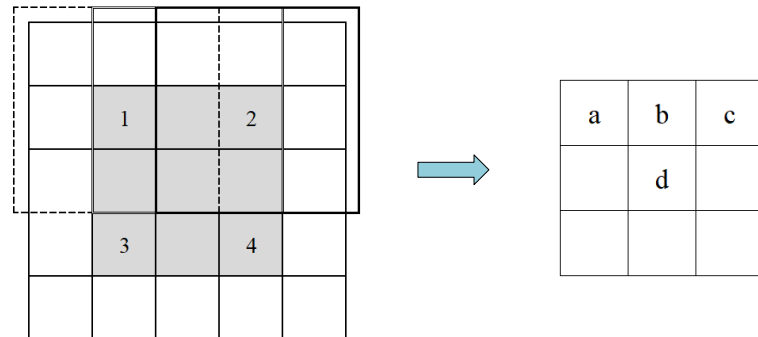
3.1.2 卷积神经网络中的常用网络层

➤ 1. 卷积层

(4) PyTorch框架中的常用卷积类型

② ConvTranspose2d

先用数值0给左边的特征图做插值，使相邻两个元素的间隔为卷积的步长值即插值的个数，同时边缘也需要进行与插值数量相等的补0。这时卷积核的滑动步长就不再是2，而是1，步长值体现在插值补0的过程中。



3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 1. 卷积层

(1) PyTorch框架中的常用卷积类型

② ConvTranspose2d

ConvTranspose2d类的语法格式如下。

torch.nn.ConvTranspose2d(in_channels, out_channels, kernel_size, stride=1, padding=0, output_padding=0, groups=1, bias=True, dilation=1, padding_mode='zeros')

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 1. 卷积层

(1) PyTorch框架中的常用卷积类型

② ConvTranspose2d

ConvTranspose2d类的语法格式如下。

ConvTranspose2d类的常用参数及其说明与Conv2d类一致。增加的参数output_padding接收int或tuple，表示对输出的hw填充的层数。

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 2. 池化层

(1) MaxPool2d

MaxPool2d类的语法格式如下。

`torch.nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1, return_indices=False, ceil_mode=False)`

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 2. 池化层

参数名称	说明
kernel_size	接收int或tuple，表示池化核尺寸，无默认值
stride	接收int或tuple，表示窗口的步幅，默认就等于窗口尺寸，默认为None
padding	接收int或tuple，表示数据hw方向上的零填充层数，默认为0
dilation	接收int或tuple，表示一个控制窗口中元素步幅的参数，默认为1
return_indices	接收bool，表示如果等于True，会返回输出最大值的序号，对于上采样操作会有帮助，默认为False
ceil_mode	接收bool，表示如果等于True，计算输出数据的hw的时候，会使用向上取整，代替默认的向下取整的操作，默认为F
kernel_size	接收int或tuple，表示池化核尺寸，无默认值

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 2. 池化层

(2) AvgPool2d

AvgPool2d类的语法格式如下。

torch.nn.AvgPool2d(kernel_size, stride=None, padding=0, ceil_mode=False, count_include_pad=True, divisor_override=None)

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 2. 池化层

(2) AvgPool2d

AvgPool2d类的常用参数及其说明如表所示。

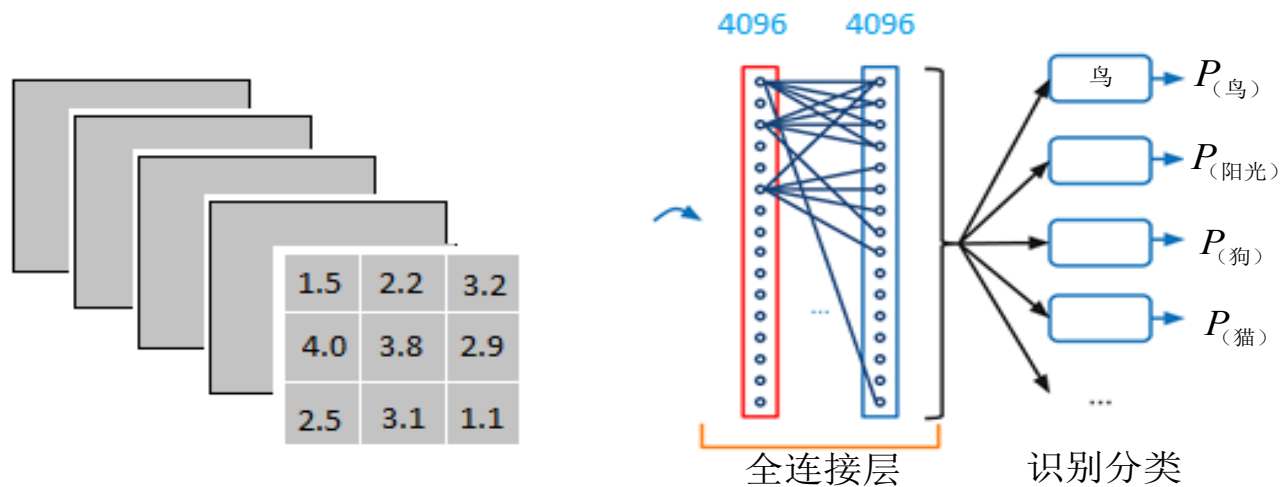
参数名称	说明
kernel_size	接收int或tuple，表示池化核尺寸，无默认值
stride	接收int或tuple，表示窗口的步幅，默认就等于窗口尺寸，默认为None
padding	接收int或tuple，表示数据hw方向上的零填充层数，默认为0
ceil_mode	接收bool，表示如果等于True，计算输出数据的hw的时候，会使用向上取整，代替默认的向下取整的操作，默认为False
count_include_pad	接收bool，表示如果等于True，计算平均池化时，将包括padding填充的0，默认为True
divisor_override	接收int，表示求平均值时，可以不使用像素值的个数作为分母，而是使用除法因子，默认为None

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 3. 全连接层

全连接层是指每个神经元与前一层所有的神经元全部连接，在整个卷积神经网络中起到“分类器”的作用，即通过卷积、激活函数、池化等深度网络提取出图像的特征后，再经过全连接层对结果进行识别分类。

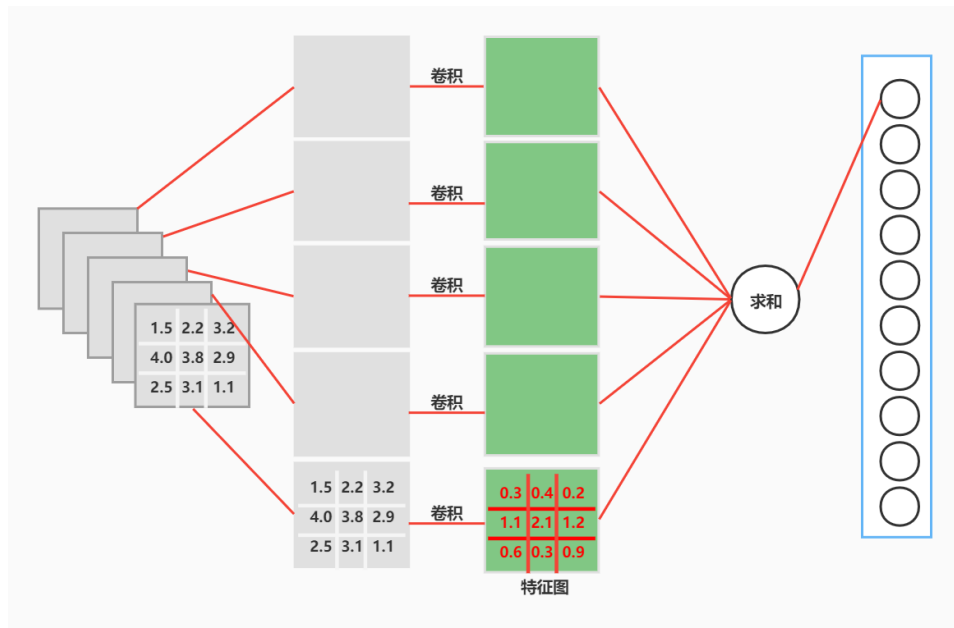


3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 3. 全连接层

要将 $3 \times 3 \times 5$ 的输出转换成 1×4096 的形式，可以理解为在中间做了一个卷积，用一个 $3 \times 3 \times 5$ 的卷积核去卷积激活函数的输出，得到的结果就是全连接层的一个神经元的输出，这个输出就是一个值。因为有4096个神经元，所以就是用一个 $3 \times 3 \times 5 \times 4096$ 的卷积层去卷积激活函数的输出。全连接层的计算过程，如图所示。



3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 3.全连接层

PyTorch的Linear类用于设置网络中的全连接层，语法格式如下。

`torch.nn.Linear(in_features, out_features, bias=True)`

Linear类的常用参数及其说明如表所示。

参数名称	说明
in_features	接收int，表示上层神经元个数，无默认值
out_features	接收int，表示本层神经元个数，无默认值
bias	接收bool类，表示偏置，形状为[out_features]。如果设置bias = False，那么该层将不会学习一个加性偏差。默认为True

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 4. 归一化层

归一化层利用小批量上的均值和标准差，不断调整网络的中间输出，从而使整个网络在各层的中间输出的数值更稳定，提高训练网络的有效性。

归一化层的主要作用是防止梯度爆炸和梯度消失。目前归一化层主要有5种：批次归一化层（Batch Normalization, BN）、横向归一化层（Layer Normalization, LN）、实例归一化层（Instance Normalization, IN）、组归一化层（Group Normalization, GN）和可切换归一化层（Switchable Normalization, SN）。

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

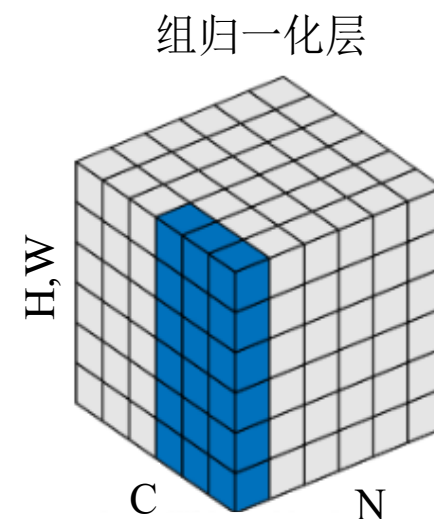
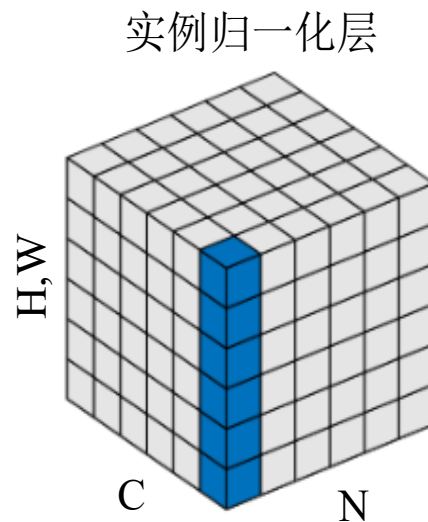
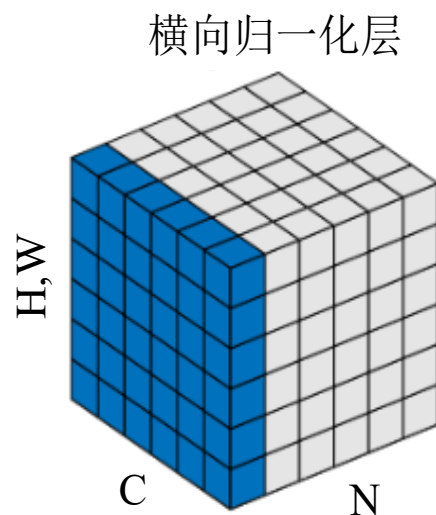
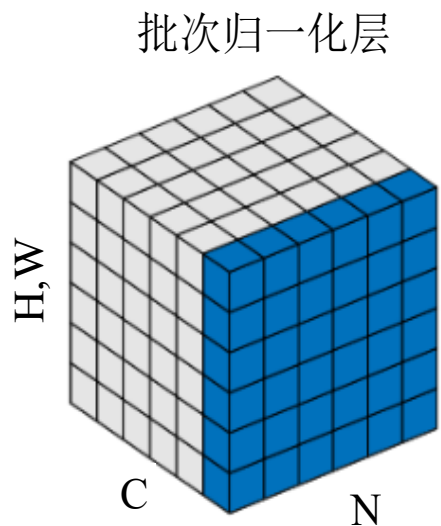
➤ 4. 归一化层

深度网络中的数据维度一般是 $[N, C, H, W]$ 或者 $[N, H, W, C]$ 格式， N 是batch size， H/W 是特征（feature）的高/宽， C 是feature的通道（channel），压缩 H/W 至一个维度，其中4种常用的归一化层的三维表示如图所示。

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 4.

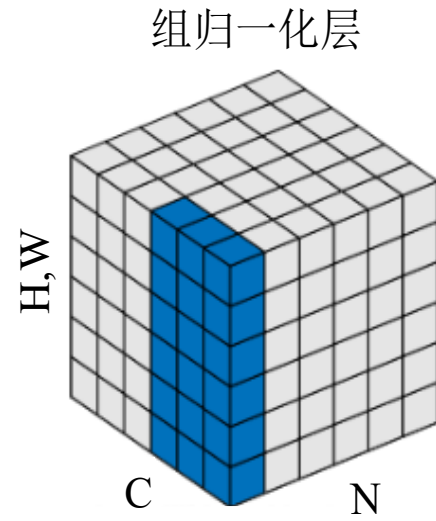
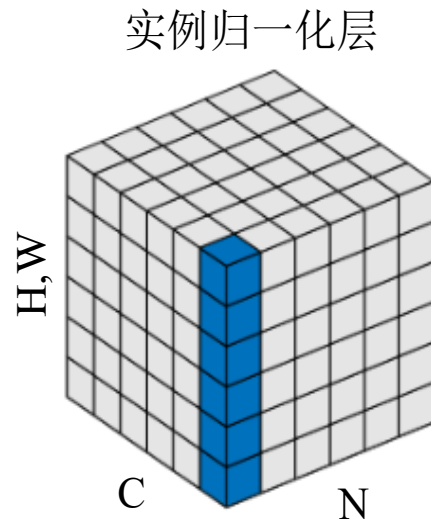
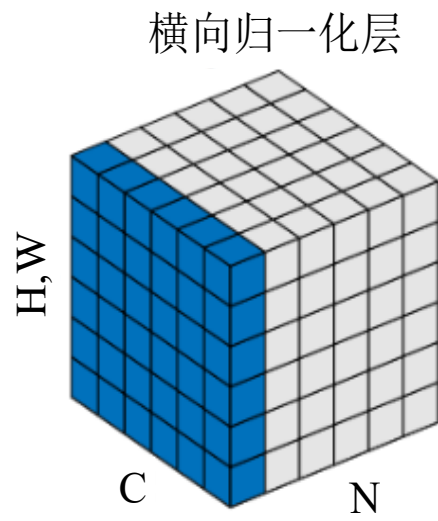
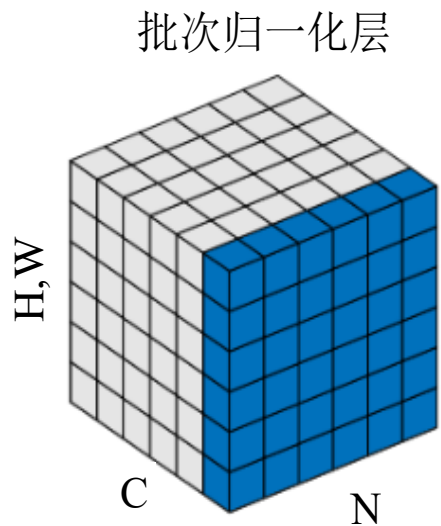


Batch Normalization层应用了一种变换规范某一层的数据，该变换可将该批所有样本在每个特征上的平均值保持在0左右，标准偏差保持在1附近，将可能逐渐将非线性传递函数（如Sigmoid函数）取值的极限饱和区靠拢的分布，

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 4.



强制拉回到均值为0方差为1的标准正态分布，使得规范化后的输出落入对下一层的神经元比较敏感的区域，以此避免梯度消失问题。因为梯度一直都能保持比较大的状态，所以对神经网络的参数调整效率比较高，即向损失函数最优值迈动的步子大，可以加快收敛速度。

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 4.归一化层

BatchNorm2d类的语法格式如下。

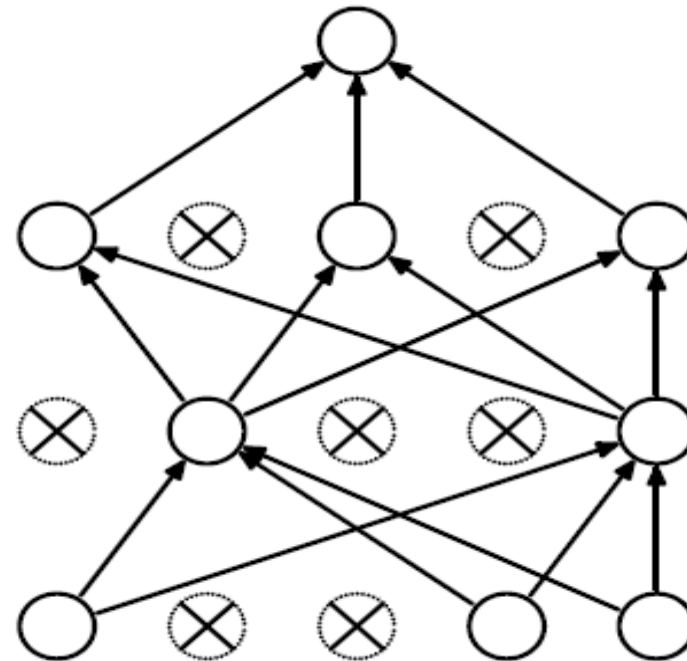
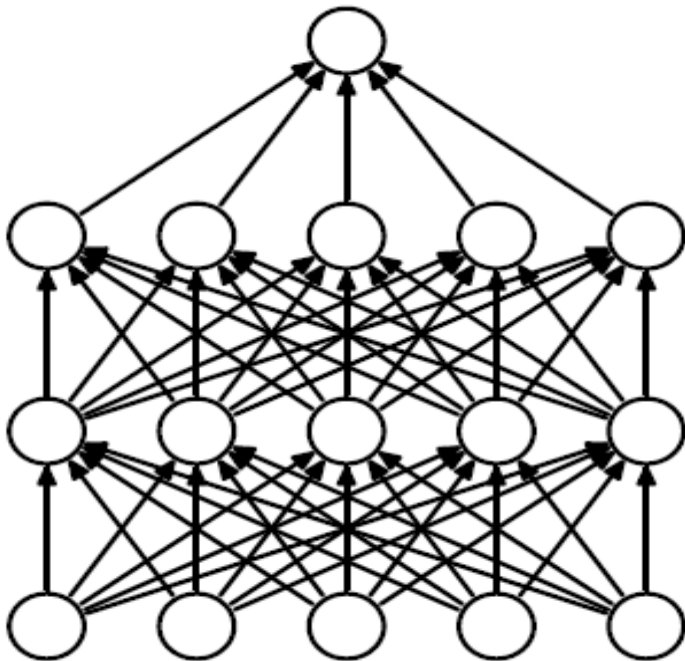
```
torch.nn.BatchNorm2d(num_features, eps=1e-05, momentum=0.1,  
affine=True, track_running_stats=True)
```

3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 5. 丢弃层

在前向传播的时候，让某个神经元的激活值以一定的概率（rate）停止工作，这样可以使模型泛化能力更强，丢弃层（Dropout）的工作原理如图所示。



3.1 卷积神经网络基础

3.1.2 卷积神经网络中的常用网络层

➤ 5. 丢弃层

Dropout类的语法格式如下。

`torch.nn.Dropout(p=0.5, inplace=False)`

Dropout类的常用参数及其说明如表所示。

参数名称	说明
p	接收float，表示张量元素被置0的概率，默认为0.5
inplace	接收int，表示是否原地执行，默认为False

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

使用MNIST数据集，是一个大型的手写体数字数据库，MNIST数据集共有训练数据60000项、测试数据10000项。每张图像的大小为 28×28 （像素），每张图像都为灰度图像。

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

➤ 1. 导入相关库

导入一些必要的包，除了常用的torch和Variable之外，还导入了DataLoader用于加载数据，使用torchvision对图像进行预处理。

```
import torch
from torch.autograd import Variable
import torchvision
from torch.utils.data import DataLoader
```

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

➤ 2. 数据加载与预处理

首先，定义超参数。

训练次数: `n_epochs`

优化器的超参数: `learning_rate`, `momentum`,

训练批次: `batch_size_train`

随机种子: `random_seed`

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

➤ 2. 数据加载与预处理

```
n_epochs = 3
batch_size_train = 64
batch_size_test = 1000
learning_rate = 0.01
momentum = 0.5
log_interval = 10
random_seed = 1
torch.manual_seed(random_seed)
```

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

➤ 2. 数据加载与预处理 (加载MNIST数据集)

```
train_loader = torch.utils.data.DataLoader(  
    torchvision.datasets.MNIST('../data/',  
                                train=True,  
                                download=False,  
                                transform=torchvision.transforms.Compose([  
                                    torchvision.transforms.ToTensor(),  
                                    torchvision.transforms.Normalize((0.1307,), (0.3081,)) ])),  
    batch_size=batch_size_train, shuffle=True)
```

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

➤ 2. 数据加载与预处理 (加载MNIST数据集)

```
test_loader = torch.utils.data.DataLoader(  
    torchvision.datasets.MNIST('../data/',  
                                train=False,  
                                download=False,  
                                transform=torchvision.transforms.Compose([  
                                    torchvision.transforms.ToTensor(),  
                                    torchvision.transforms.Normalize((0.1307,), (0.3081,)) ])),  
    batch_size=batch_size_test, shuffle=True)
```

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

➤ 2. 数据加载与预处理

以test_loader为例，观察数据集的基本概貌，其中，example_targets是图像实际对应的数字标签。

```
examples = enumerate(test_loader)
batch_idx, (example_data, example_targets) = next(examples)
print(example_targets)
print(example_data.shape)
```

3.1 卷积神经网络基础

```
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams['font.family']='SimHei'
fig = plt.figure()
for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.tight_layout()
    plt.imshow(example_data[i][0], cmap='gray', interpolation='none')
    plt.title('目标数据标签: {}'.format(example_targets[i]))
    plt.xticks([])
    plt.yticks([])
plt.show()
```

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

➤ 2. 数据力

最后，使用

目标数据标签：3



目标数据标签：9



目标数据标签：4



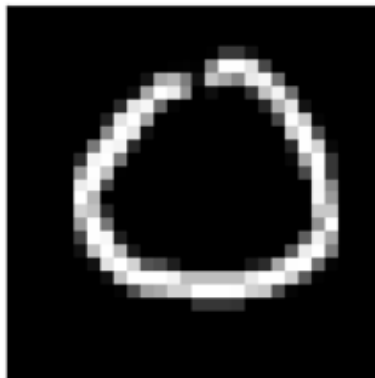
目标数据标签：9



目标数据标签：9



目标数据标签：0



3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

➤ 3. 构建网络

构建卷积神经网络时，使用两个Conv2d卷积层和两个全连接层Linear，并且选择整流线性单元（简称ReLU）作为激活函数以及使用两个Dropout层。

首先，导入相关库，如代码3-14所示。

```
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
```

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

- 3. 构建网络：然后，为构建的网络创建一个新类Net

```
class Net(nn.Module):
```

```
    def __init__(self):
```

```
        super(Net, self).__init__()
```

```
        self.conv1 = nn.Conv2d(1, 10, kernel_size=5)
```

```
        self.conv2 = nn.Conv2d(10, 20, kernel_size=5)
```

```
        self.conv2_drop = nn.Dropout2d()
```

```
        self.fc1 = nn.Linear(320, 50)
```

```
        self.fc2 = nn.Linear(50, 10)
```

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

➤ 3. 构建网络：然后，为构建的网络创建一个新类Net

```
def forward(self, x):  
    x = F.relu(F.max_pool2d(self.conv1(x), 2))  
    x = F.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)), 2))  
    x = x.view(-1, 320)  
    x = F.relu(self.fc1(x))  
    x = F.dropout(x, training=self.training)  
    x = self.fc2(x)  
    return F.log_softmax(x)
```

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

➤ 3. 构建网络：初始化网络和优化参数

```
network = Net()  
optimizer=optim.SGD(network.parameters(), lr=learning_rate,  
momentum=momentum)
```

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

➤ 4. 训练网络

在训练网络过程中，分别定义训练函数和测试函数。通过训练函数，观察使用随机初始化的网络参数获得的精度和损失率。在测试函数中跟踪正确分类的数字来计算网络的精度。首先，设置参数train_losses、train_counter、test_losses和test_counter并初始化，

```
train_losses = []  
train_counter = []  
test_losses = []  
test_counter = [i * len(train_loader.dataset) for i in range(0, n_epochs + 1)]
```



3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

➤ 4. 训练网络：进行一次循环，

```
def train(epoch):  
    network.train()  
    for batch_idx, (data, target) in enumerate(train_loader):  
        optimizer.zero_grad()  
        output = network(data)  
        loss = F.nll_loss(output, target)  
        loss.backward()  
        optimizer.step()
```

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

```
if batch_idx % log_interval == 0:
    print('Train Epoch: {} [{} / {} ( {:.0f}% )]\tLoss: {:.6f}'.format(
        epoch, batch_idx * len(data), len(train_loader.dataset),
        100. * batch_idx / len(train_loader), loss.item()))
    train_losses.append(loss.item())
    train_counter.append(
        (batch_idx * 64) + ((epoch-1) * len(train_loader.dataset)))
    torch.save(network.state_dict(), '../tmp/model.pth')
    torch.save(optimizer.state_dict(), '../tmp/optimizer.pth')

train(1)
```

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

➤ 4. 训练网络：过

Train Epoch: 1 [44800/60000 (75%)]	Loss: 0.575355
Train Epoch: 1 [45440/60000 (76%)]	Loss: 0.479146
Train Epoch: 1 [46080/60000 (77%)]	Loss: 0.548565
Train Epoch: 1 [46720/60000 (78%)]	Loss: 0.577999
Train Epoch: 1 [47360/60000 (79%)]	Loss: 0.520859
Train Epoch: 1 [48000/60000 (80%)]	Loss: 0.451427
Train Epoch: 1 [48640/60000 (81%)]	Loss: 0.448887
Train Epoch: 1 [49280/60000 (82%)]	Loss: 0.857061
Train Epoch: 1 [49920/60000 (83%)]	Loss: 0.748952
Train Epoch: 1 [50560/60000 (84%)]	Loss: 0.400014
Train Epoch: 1 [51200/60000 (85%)]	Loss: 0.700703
Train Epoch: 1 [51840/60000 (86%)]	Loss: 0.723266
Train Epoch: 1 [52480/60000 (87%)]	Loss: 0.446432
Train Epoch: 1 [53120/60000 (88%)]	Loss: 0.515988
Train Epoch: 1 [53760/60000 (90%)]	Loss: 0.536006
Train Epoch: 1 [54400/60000 (91%)]	Loss: 0.373517
Train Epoch: 1 [55040/60000 (92%)]	Loss: 0.401741
Train Epoch: 1 [55680/60000 (93%)]	Loss: 0.392790
Train Epoch: 1 [56320/60000 (94%)]	Loss: 0.933339
Train Epoch: 1 [56960/60000 (95%)]	Loss: 0.453359
Train Epoch: 1 [57600/60000 (96%)]	Loss: 0.437842
Train Epoch: 1 [58240/60000 (97%)]	Loss: 0.513740
Train Epoch: 1 [58880/60000 (98%)]	Loss: 0.570341
Train Epoch: 1 [59520/60000 (99%)]	Loss: 0.316010

```
def test():
```

```
    network.eval()
```

```
    test_loss = 0
```

```
    correct = 0
```

```
    with torch.no_grad():
```

```
        for data, target in test_loader:
```

```
            output = network(data)
```

```
            test_loss += F.nll_loss(output, target, size_average=False).item()
```

```
            pred = output.data.max(1, keepdim=True)[1]
```

```
            correct += pred.eq(target.data.view_as(pred)).sum()
```

```
    test_loss /= len(test_loader.dataset)
```

```
    test_losses.append(test_loss)
```

```
    print('\nTest set: Avg. loss: {:.4f}, Accuracy: {} / {} ({:.0f}%) \n'.format(
```

```
        test_loss, correct, len(test_loader.dataset),
```

```
        100. * correct / len(test_loader.dataset)))
```

```
test():
```

```
Test set: Avg. loss: 0.1742, Accuracy: 9496/10000 (95%)
```

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

➤ 4. 训练网络

最后，循环n_epochs次训练进行验证，
方法test(),

```
Train Epoch: 3 [49200/60000 (82%)] Loss: 0.330000
Train Epoch: 3 [49920/60000 (83%)] Loss: 0.198462
Train Epoch: 3 [50560/60000 (84%)] Loss: 0.332324
Train Epoch: 3 [51200/60000 (85%)] Loss: 0.286326
Train Epoch: 3 [51840/60000 (86%)] Loss: 0.299743
Train Epoch: 3 [52480/60000 (87%)] Loss: 0.120664
Train Epoch: 3 [53120/60000 (88%)] Loss: 0.175167
Train Epoch: 3 [53760/60000 (90%)] Loss: 0.156284
Train Epoch: 3 [54400/60000 (91%)] Loss: 0.316813
Train Epoch: 3 [55040/60000 (92%)] Loss: 0.337862
Train Epoch: 3 [55680/60000 (93%)] Loss: 0.173838
Train Epoch: 3 [56320/60000 (94%)] Loss: 0.473166
Train Epoch: 3 [56960/60000 (95%)] Loss: 0.135538
Train Epoch: 3 [57600/60000 (96%)] Loss: 0.192759
Train Epoch: 3 [58240/60000 (97%)] Loss: 0.241007
Train Epoch: 3 [58880/60000 (98%)] Loss: 0.231019
Train Epoch: 3 [59520/60000 (99%)] Loss: 0.115733
```

```
Test set: Avg. loss: 0.0854, Accuracy: 9720/10000 (97%)
```

test()

for epoch in range(1, n_epochs + 1):

 train(epoch)

 test()

3.1 卷积神经网络基础

3.1.3 基于卷积神经网络的手写数字识别

```
import matplotlib.pyplot as plt

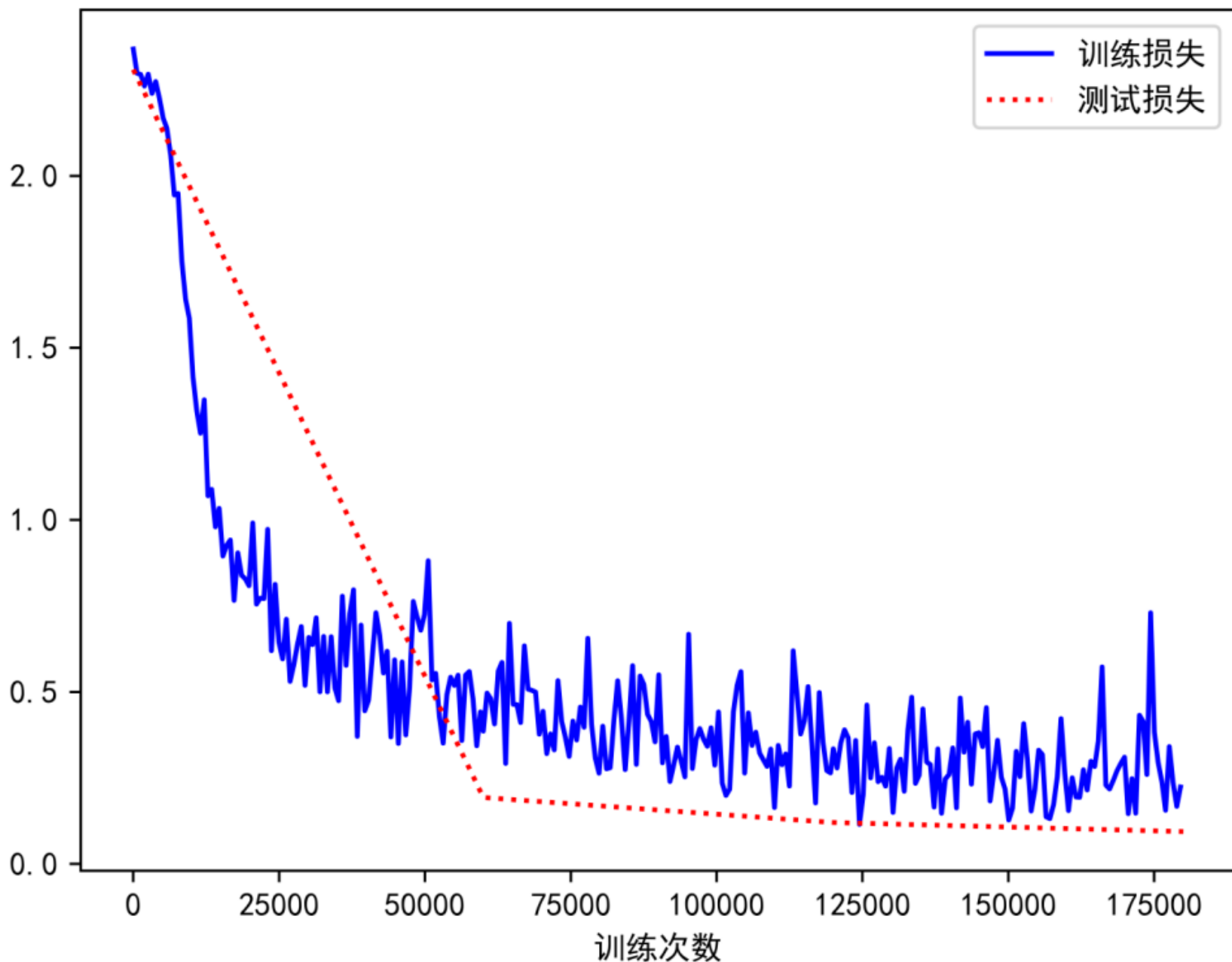
fig = plt.figure()
plt.plot(train_counter, train_losses, color='blue')
plt.plot(test_counter, test_losses, color='red', linestyle=':')
plt.legend(['训练损失', '测试损失'], loc='upper right')
plt.xlabel('训练次数')
plt.ylabel('损失')
plt.savefig('../tmp/1.png', dpi=1080)
plt.show()
```

3.1 卷积

3.1.3 卷积核

➤ 5. 性能

通率。下面损失能进行评



97%的准确
，对模型性

3.1 卷积神经网络基础

```
examples = enumerate(test_loader)
batch_idx, (example_data, example_targets) = next(examples)
with torch.no_grad():
    output = network(example_data)
fig = plt.figure()
for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.tight_layout()
    plt.imshow(example_data[i][0], cmap='gray', interpolation='none')
    plt.title('预测结果: {}'.format(
        output.data.max(1, keepdim=True)[1][i].item()))
    plt.xticks([])
    plt.yticks([])
```

3.1 卷积神经网络基础

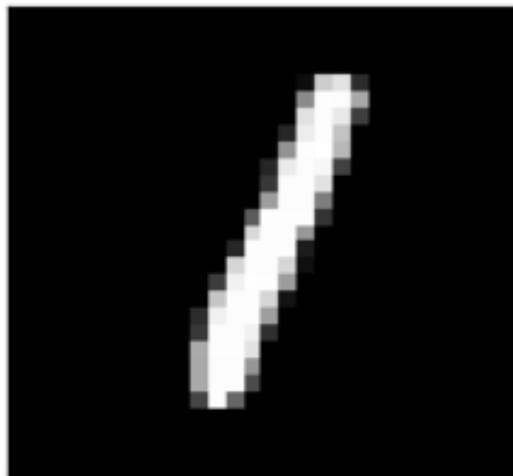
3.



预测结果：6



预测结果：1



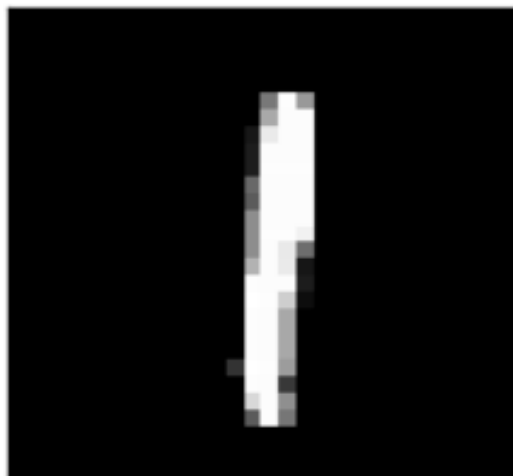
预测结果：7



预测结果：0



预测结果：1



预测结果：2

